
Green Energy Monitoring Platform

Engineering Technical Report

*Design, implementation, measurement and operation of a
budget-constrained retrofit decision-support platform
for public-sector building portfolios*

Authors: Ahmed Ebaid (platform, optimiser, infrastructure)
Nada Wagdy (catalogue, parameters, citations)

Supervisor: Prof. Hala Moshier

Repository: AhmadEbaid001/EcoForge (private)

Reported revision: 43ef04f, 12 September 2026

Commits covered: 132, a05a507 → 43ef04f

Period: 14 August — 12 September 2026

This report documents what was built, what was measured, and why each decision was taken rather than its alternative. Three of its principal results contradict the project's own proposal and are reported as such.

September 2026

Executive summary

Egypt is commissioning public buildings faster than it can make them efficient, and the electricity reaching them is not clean: the national utility’s own annual report puts new and renewable generation at 11.65% of the total and the environmental indicator for the thermal plants supplying the remainder at 380.3 t CO₂/GWh. The fifty real, named public buildings this project is measured on — ministries, hospitals, schools, courts and libraries across Greater Cairo — consume 132.7 GWh a year between them.

The engineering of a retrofit is not the difficulty. The difficulty is that nobody can retrofit everything at once, so a finite budget has to be committed to particular buildings and particular measures, and that commitment is normally made without per-building consumption data and without a common basis on which measures can be compared. GEMP is the system built to make that commitment measurable, arguable and re-checkable.

What was built

Five containers on one host: a self-hosted MQTT broker, a TimescaleDB store, an nginx edge, a simulator publishing per-building readings in the shape a physical clamp meter would send, and one application process containing six modules. Readings are HMAC-signed and hash-chained as they land, with chain heads anchored to a file outside the database volume. A nightly refit learns each building’s hourly load shape and rewrites the annual figure the optimiser costs against. The allocation itself is formulated as a multiple-choice knapsack and solved to proven optimality by CP-SAT against three baselines, under the budget and under a per-district cap. A front end written as vanilla ES modules — no framework, no build step, no CDN, no tile server — renders nine screens in English and Arabic, in light and dark, and works with the network cable unplugged.

What was measured

- **The exact solver’s unconstrained advantage is small, and is reported as small.** Against a correctly repaired greedy baseline it wins by a median of +1.28% over twenty budgets. It earns its place through the two-per-district cap no greedy rule can express, where the median is +8.63% and the advantage *grows* with the budget to +14.77% at 200 MEGP.
- **Against the status-quo policy the margin is large.** At 50 MEGP CP-SAT delivers 170.2 M kgCO₂e of life-cycle benefit against equal split’s 39.6 M — and it does so by funding fifteen buildings where the heuristics fund forty-two.
- **Every instance solved to proven optimality:** 120/120 OPTIMAL, slowest solve 197 ms.
- **The life-cycle layer changes almost no decisions.** At 16 of 20 budgets the carbon and raw-kWh objectives fund the same set; the worst carbon cost of ignoring the layer is 0.01%. This contradicts the proposal and is reported.
- **Rooftop photovoltaics is never the densest single measure** at any of the fifty buildings. LED lighting wins at 28, zone control at 21, insulation at 1. This also contradicts

the proposal.

- **Forecasting** reads a median held-out MAPE of 3.4% against a seasonal-naive 5.8%; **anomaly detection** reads precision 0.80 at recall 0.89, against review gates of 0.6 and 0.8.

What distinguishes it

Three properties, in the order the authors would defend them.

Provenance is part of the method. Every physical and economic parameter carries a source, a locator inside that source, and the date it was retrieved. The one input for which no public source exists — a 24-hour marginal emission profile for the Egyptian grid — is labelled a placeholder rather than given a citation that does not support it. That this is not clerical is itself a measured result: replacing a single previously uncited constant with a sourced one moved the embodied-carbon share of avoided emissions from a median 8.1% to 9.5% and the number of budgets at which two objectives disagree from two to four.

The claims are falsifiable and re-measured. Fourteen numbered claims carry gates fixed before measurement; a harness re-measures them on every commit and exits non-zero when one stops holding. It found the greedy-saturation defect, a mis-specified bundling claim and two anomaly-scoring artefacts — each of which had already been written down as a result before the harness contradicted it.

The deployment is defended rather than assumed. 548 automated tests, a security gate that blocks on any finding from any of seven scanners, keyless image signing, and a host that health-checks a release and puts the previous one back by itself. A deliberately broken image was deployed to prove the rollback path works.

What is not claimed

The readings are simulated. The saving fractions are literature values, not measured retrofit outcomes for these buildings. Lifetime carbon is a projection, not a measurement. A verified chain says nobody altered what the meter sent, not that the meter was right. One parameter is an acknowledged placeholder and two are market estimates inside a stated bracket. Each of these is stated on the screen that presents the affected number, not only here.

Contents

Executive summary	i
1 Mandate, problem and constraints	1
1.1 The competition context	1
1.2 The problem, stated precisely	1
1.3 Why the naive formulation fails	2
1.4 Constraints that shaped the build	2
1.5 The cut list, fixed in advance	2
1.6 How to read this report	3
2 The review that shaped the project	4
2.1 Why there is a review chapter at all	4
2.2 The findings, and what became of each	4
2.3 The three critical findings in detail	5
2.3.1 F1 — maximising a quantity that cannot be summed	5
2.3.2 F2 — a ranking reversal that was an accounting artefact	5
2.3.3 F3 — the optimiser’s principal input was unspecified	6
2.4 The findings that changed the evaluation	6
2.4.1 F8 — the headline comparison was weaker than it looked	6
2.4.2 F9 — a statistic with a breakdown point of zero	7
2.5 Scope, schedule and credibility	7
2.6 The verification gates the review set	7
3 Architecture	9
3.1 One process, six modules, five containers	9
3.2 The five stages and the one loop	9
3.3 What crosses each boundary	10
3.4 Module inventory	10
3.5 Three decisions that look like details	10
3.5.1 The scheduler must be in-process	10
3.5.2 The application refuses to start under more than one worker	10
3.5.3 No PostGIS, and Timescale objects outside Alembic	11
4 Data and provenance	12
4.1 The contract between two people	12
4.2 The portfolio, and two earlier versions that were wrong	12
4.2.1 What the portfolio looks like	13
4.3 The intervention catalogue	14
4.4 Parameters, and where each one comes from	15
4.4.1 The grid factor is an operating margin, not a grid average	15
4.4.2 Solar yield and orientation, measured rather than assumed	15

4.4.3	The placeholder, and why the negative result is the finding	16
4.4.4	Condition multipliers are declared assumptions	17
4.5	Validation, as a gate rather than a convention	17
5	The decision model	18
5.1	From a meter reading to a per-intervention saving (F3)	18
5.1.1	Generation takes a separate path	18
5.2	A common horizon, and embodied carbon (F2)	19
5.3	Bundling composes multiplicatively within an end use (F4)	19
5.4	The allocation problem (F1, F8)	20
5.4.1	Integrity, and why rounding is safe here	21
5.4.2	Dominance pruning	21
5.5	Time-of-use marginal weighting	21
6	Results	23
6.1	Allocation quality at one budget	23
6.2	What the budget buys, across the whole range	23
6.3	Where exact optimisation earns its place	23
6.3.1	Solver performance	25
6.4	A negative result: the life-cycle layer changes almost no decisions	25
6.5	A second negative result: no portfolio-wide priority list is correct	26
6.6	The claims harness output	26
6.7	Results that require the metered series	27
6.7.1	The metered series reaches the allocation decision (F15)	27
6.7.2	Forecasting (F3)	28
6.7.3	Anomaly detection (F9)	28
6.7.4	Integrity (F5-b)	28
7	Forecasting and anomaly detection	29
7.1	Why there is a forecaster at all	29
7.2	Model choice (F10)	29
7.3	Features, and the one that mattered most	29
7.3.1	Predicting the ratio, not the load	30
7.4	Anomaly detection (F9)	30
7.4.1	A robust statistic, and a relative residual	30
7.4.2	Three floors, each protecting against a different degenerate case	31
7.4.3	A separate detector for a stuck meter	31
7.5	Scoring: the unit is the alert	31
7.6	The threshold study	32
7.6.1	A negative result that was corrected rather than deleted	32
7.7	Two detectors, one statistic	33
8	Tamper-evident storage	35
8.1	The threat, stated narrowly	35
8.2	Chaining, at the same cost per row	35
8.3	Canonicalisation is where silent failure lives	35
8.4	The sequence is the ingester's to assign	36
8.5	Verification: three checks, and the order matters	36
8.5.1	The verdict names the likely cause	37
8.6	What this does not claim	37
8.7	Where the key lives, and what that costs	38

9	The web application	39
9.1	The constraint that produced the architecture	39
9.2	The shell is deliberately empty	39
9.3	The nine screens	39
9.4	The map, and why it has no tile server	41
9.5	Charts drawn by hand, and what a reader can do with them	41
9.6	Two appearances, and tokens written three times	42
9.7	Two languages, and the defect that shipped	43
9.8	The interface’s contract, enforced by tests	43
9.9	The phone, which found nine defects in one afternoon	44
9.10	Accessibility work that is not decoration	44
10	Security architecture	45
10.1	The shape of the thing	45
10.2	Sessions, not tokens	45
10.3	Passwords	46
10.4	Login as an enumeration and denial-of-service surface	46
10.5	CSRF, and one deliberate exemption	47
10.6	Roles, and the audit trail	47
10.7	Everything else the middleware does	48
10.8	The independent security review of 12 September	48
10.8.1	What the live host verified	48
11	The Judge Pass	50
11.1	The problem this solves	50
11.2	One shared code, and where the limits live	50
11.3	The flow, and two details that matter	51
11.4	What the account is	51
11.4.1	The generated password	51
11.5	The email	52
11.5.1	Three transports, and why there are three	52
11.6	The card	52
11.7	Operational state, and what it cost to verify	52
12	Infrastructure and deployment	54
12.1	Five containers, one host, one file	54
12.2	The host	55
12.2.1	Reaching it without opening a port	55
12.3	Bootstrap: what a fresh host needs	55
12.4	Deploying a release	56
12.4.1	What counts as “it works”	57
12.4.2	The privilege boundary	57
12.5	Rebuilding the history	57
12.6	Backups, and the drill that found a real defect	58
12.7	The drill that needs no server	58
13	The pipeline	60
13.1	One workflow, three jobs, ordered by how fast they fail	60
13.2	The security policy: block on anything	60
13.2.1	The one thing that does not block	61
13.2.2	Getting past a finding	61
13.3	Supply chain	61

13.4	Deploy, and the human gate	62
13.5	The same script for a person and for the pipeline	62
13.6	Known gaps, stated rather than discovered	62
14	Testing	64
14.1	What the suite is and is not	64
14.2	Fixtures are built in code, never read from the data files	64
14.3	The suite enforces what the database enforces	65
14.4	The four gates, runnable locally	65
14.5	Tests that exist because of a specific defect	65
15	The evaluation harness	67
15.1	A claim is not a test	67
15.2	How the grid is built	67
15.3	Three properties that make the output worth reading	68
15.4	What it caught	68
15.5	Exit status is the product	68
16	Engineering chronology	70
16.1	Shape of the work	70
16.2	Phase by phase	70
16.3	The week the deadline moved	71
16.4	Ten commits on 20 August	71
16.5	How the work was handed over	72
17	Defects and lessons	73
17.1	The governing pattern	73
17.2	Time, and a clock that is not the wall clock	73
17.3	Databases	74
17.4	Containers	74
17.5	The browser	75
17.6	Measurement	75
17.7	Process	76
17.8	The shape of a good failure	77
18	Limitations and future work	78
18.1	Limitations, stated as the screens state them	78
18.2	Threats to validity	78
18.3	Technologies evaluated and rejected	79
18.4	Future work	80
18.4.1	Closing the layers that are shallower than they look	80
18.4.2	Operational	80
18.4.3	What would make it a product rather than a demonstration	80
A	Appendices	81
A.1	Reproducing every number in this report	81
A.2	Configuration reference	81
A.3	Data model	83
A.4	The claims, with their gates	84
A.5	Repository inventory	84
A.6	Sources for the parameters and the catalogue	84

List of Figures

3.1	The five stages and the one feedback loop	9
3.2	Deployed components and what crosses each boundary	10
4.1	Portfolio composition	13
5.1	The time-of-use marginal profile	22
6.1	What the budget buys, by allocation method	24
6.2	Why a single median hides the argument	24
7.1	The precision/recall curve traced by the threshold	33
8.1	The hash chain and its external anchor	35
9.1	Four of the nine screens	40
9.2	The alert inbox and the administration screen	40
16.1	Commits per week	70

List of Tables

1.1	Constraints, and what each one forced.	2
2.1	The fourteen findings and their disposition. “Effort” is the review’s own estimate; total code-side effort was assessed at roughly 22 hours.	4
2.2	The review’s verification gates, and the measured outcome. Sources: <code>out/evaluation/claims.csv</code> and the measured-data runs of Chapter 6.	7
3.1	The application’s modules, with what each owns. Line counts are from <code>git ls-files</code> ; the repository holds 174 tracked files and roughly 42 300 lines across code, tests, front end and infrastructure.	11
4.1	Portfolio statistics, measured from the fixture. Operational attributes (occupancy, HVAC age, insulation quality, consumption) are synthetic and declared as such: no open dataset provides them for Egyptian public buildings, and a real deployment would collect them through a profile-entry form.	14
4.2	The catalogue. Every row carries a citation; an uncited row fails <code>python -m gemp.domain.catalog --validate --strict</code> , which gates submission.	14
4.3	Parameters on which published results depend. All sources retrieved 7 September 2026. Each reference names a page, chapter, article or query rather than a document.	15
4.4	Roof orientation factors, measured at 28° slope and normalised to due south. PVGIS azimuth convention: 0° south, negative east, positive west.	16
6.1	The four solvers at a budget of 50,000,000 EGP, objective <code>lca_carbon</code> . Transcribed from <code>out/evaluation/optimizer_sweep.csv</code>	23
6.2	All fourteen claims, transcribed verbatim from <code>out/evaluation/claims.csv</code> . Gates were fixed before measurement. F15 reports SKIP in the file-only run by design and PASS with a database attached (§6.7).	27
7.1	Threshold sweep behind the choice of k , from <code>python -m gemp.ml.evaluate</code> . Measured on an earlier portfolio and reported for the shape of the trade-off rather than as a current figure; the current operating figures are in §6.7.	32
10.1	Security review findings. Severity reflects real-world reachability at the time of review, not textbook category.	49
14.1	Test functions per file. The collected count is higher because fourteen parametrised decorators expand some functions into many cases — <code>test_prune.py</code> is 8 functions and 55 cases.	64
16.1	The phases, with the commits that opened and closed each.	70

Chapter 1

Mandate, problem and constraints

1.1 The competition context

GEMP is Team Ecoforge’s entry to the RoboDam 2026 Smart Systems track. The team is two people with a deliberate division of labour: Ahmed Ebaid owns the platform, the optimiser and the infrastructure; Nada Wagdy owns the intervention catalogue, the physical parameters and their citations. The supervisor is Prof. Hala Moshier.

Two dates govern everything in this report. Preliminary materials — presentation, poster and a three-minute video — were submitted on **9 September 2026**. Final judging is in person at a booth on **14 September 2026**. An earlier deadline of 31 August moved to 9 September during the project, which bought nine days; how those days were spent is the subject of Chapter 16.

1.2 The problem, stated precisely

A public authority holds a portfolio of buildings and a capital budget for one financial year. Each building admits several possible retrofits — insulation, glazing, efficient cooling plant, zone control, lighting, rooftop generation — and combinations of them. Each option has a capital cost, an annual energy saving, an embodied carbon burden and a service life. The authority must decide *which buildings receive which measure*, subject to the budget and to the constraints procurement actually imposes.

Three properties separate a decision-support system that is useful here from one that merely looks quantitative, and the whole of this report is an account of taking them seriously.

- (i) **The comparison must be like-for-like.** Measures differ in service life, in the end use they act on, and in the carbon embodied in their manufacture. Crediting each measure over its own lifetime — as the project’s own proposal did — produces ranking reversals that are artefacts of the accounting rather than properties of the buildings.
- (ii) **The allocation must respect real constraints.** A budget is one. A cap on how many buildings may be funded in one district is another, and it is the one a greedy rule cannot express at all.
- (iii) **The claims must be falsifiable and re-measured.** A system whose documentation quotes numbers its own software no longer produces is worse than one that quotes none.

This is a budget allocator, not a monitoring dashboard.

The distinction governs every design decision that follows. Ingestion, forecasting, anomaly detection, the map and the dashboards exist to feed one decision or to display it: which buildings get funded, and why those and not the others. A feature that serves neither purpose was cut, and the cut list was fixed in advance (§1.5) so that schedule slippage cost peripheral features rather than triggering a negotiation mid-sprint.

1.3 Why the naive formulation fails

It is worth stating the failure the project was corrected *out of*, because it is the natural first attempt and it is wrong in a way that produces plausible numbers.

The proposal defined a life-cycle-adjusted score as a ratio — benefit per thousand EGP — and then maximised the sum of those ratios subject to the budget. Benefit densities are not additive. The sum of two ratios with different denominators has no physical interpretation, the reported objective value carries no unit, and the resulting programme is degenerate: it rewards selecting many cheap high-ratio items irrespective of the absolute benefit delivered. The allocation produced is not the allocation the project claimed to compute.

That was the first of fourteen findings from an architectural review of the proposal, conducted on 14 August 2026 before any implementation existed. Chapter 2 reproduces the findings and records what was done about each one. The review is the spine of this report: nine of its fourteen findings are labelled in the source at the module that resolves them, and the claim identifiers used throughout (**F1**, **F5**, **F8**...) are its numbering.

1.4 Constraints that shaped the build

Table 1.1: Constraints, and what each one forced.

Constraint	Consequence for the design
Two people, seventeen days at the time of review	The optimiser was built first, against CSV fixtures, with no infrastructure behind it. If the pipeline slipped, the artefact being judged still ran.
The demonstration may have no network	No CDN, no web font, no map tile server, no charting library. A Content-Security-Policy of <code>default-src 'self'</code> enforces this at the browser rather than trusting future discipline.
Managed cloud free tiers had already failed twice	One Compose file, five containers, one host. The laptop is the primary demonstration host; the cloud VM is a deployment claim, not a dependency.
The catalogue's <i>values</i> belong to the other author	Only the catalogue's <i>schema</i> sits on the critical path. Two version-controlled files carry the entire interface between the two people, and both are validated in continuous integration.
No hardware exists	The MQTT topic contract is the hardware seam. A physical node publishes the same message to the same topic; nothing downstream can tell the difference. The VPN tier the proposal specified for that hardware was cut and is described as designed-for rather than built.
Judging happens at a booth, on the judges' own phones	The platform had to work on a real phone, and a visitor with no account had to be able to get in. Chapters 9 and 11.

1.5 The cut list, fixed in advance

The review required an explicit ordering of what would be abandoned under schedule pressure, so that slippage cost peripheral features rather than the core deliverable. The list, in the order things would go:

SMTP alerting → WireGuard VPN → Grafana → anomaly detection → forecasting

The first three were cut. Anomaly detection and forecasting survived and both shipped. **The optimiser, the map and the evaluation harness were never cuttable.** What each cut cost and why it was still the right call is recorded in §18.3.

1.6 How to read this report

Chapter 2 is the review that shaped the project. Chapter 3 gives the architecture and the one loop in it. Chapter 4 covers the portfolio, the catalogue and the provenance of every parameter. Chapter 5 develops the decision model as equations. Chapter 6 reports what solving it produced, including three negative results. Chapters 7 to 11 cover the learning layer, the integrity chain, the front end, the security architecture and the Judge Pass. Chapters 12 to 15 cover deployment, the pipeline, the test suite and the evaluation harness. Chapter 16 is the engineering chronology. Chapter 17 is the defect record — the traps that cost real time — which is the chapter the authors would hand to a successor first.

Throughout, a marked aside carries the reasoning behind a choice:

The form of these asides is deliberate.

The project’s house style is that comments explain reasoning rather than mechanics, and that when a measurement contradicts an earlier claim the claim is corrected in writing rather than quietly dropped. This report follows the same rule: where a number here supersedes one the team published earlier, both appear and the reason for the change is given.

Chapter 2

The review that shaped the project

2.1 Why there is a review chapter at all

On 14 August 2026 the proposal was subjected to an architectural review before any implementation existed. That timing is the point. The review’s own baseline observation records it plainly: the proposal scheduled Week 1 — provisioned server, Compose skeleton, schemas, sensor simulator, submission API — as complete by 14 August, and on that date the repository was empty. Seventeen days remained against a plan written for twenty-four, with one implementer.

Fourteen findings came out of it, graded by consequence:

- **Critical** — invalidates a central technical claim, or puts the core deliverable at risk. Fix before submission.
- **High** — a reviewer is likely to find it; damages credibility or costs schedule time.
- **Medium** — degrades quality or wastes effort.
- **Low** — hygiene.

Every claim identifier used in this report and in the software — **F1**, **F5**, **F8-b** — is this numbering. Nine of the fourteen findings are labelled in the source at the module that resolves them, so that the code can be pointed at when the finding is questioned.

2.2 The findings, and what became of each

Table 2.1: The fourteen findings and their disposition. “Effort” is the review’s own estimate; total code-side effort was assessed at roughly 22 hours.

ID	Finding	Area	Sev.	Disposition
F1	Optimiser maximises a sum of benefit <i>densities</i> ; densities are not additive	Formulation	Critical	Fixed. optimize/ilp.py
F2	Life-cycle comparison uses inconsistent horizons; the headline reversal is an artefact	Formulation	Critical	Fixed. domain/lifecycle.py
F3	No model links a whole-building meter reading to per-intervention savings	Data model	Critical	Fixed. domain/savings.py
F4	Interventions treated as additive; savings on a shared end use do not stack	Formulation	High	Fixed. domain/candidates.py
F5	Per-row HMAC does not detect record <i>deletion</i> or truncation	Integrity	High	Fixed. ingest/integrity.py
F6	Cold start: forecasts and anomaly baselines do not exist at boot	Operations	High	Fixed. seed.py

ID	Finding	Area	Sev.	Disposition
F7	Schedule baseline slipped a week; plan assumes 24 days, 17 remain	Delivery	High	Re-baselined; optimiser built first
F8	Headline claim rests on ILP vs greedy, where the gap is inherently small	Evaluation	High	Fixed <i>and</i> the claim rewritten. Ch. 6
F9	Anomaly threshold uses mean and standard deviation; anomalies mask each other	Detection	Medium	Fixed. <code>ml/anomaly.py</code>
F10	Prophet is a heavy dependency, weak on hourly load, unnecessary	Stack	Medium	Rejected. <code>ml/forecast.py</code>
F11	Scope includes components that demonstrate nothing (VPN, SMTP, six services)	Scope	Medium	Cut. <code>api/main.py</code> , <code>ingest/webhook.py</code>
F12	Money units mix EGP and thousands of EGP along the scoring path	Correctness	Medium	Fixed. EGP integers throughout
F13	Live demonstration depends on a remote VM and remote map tiles	Delivery	High	Fixed. Offline guarantee, Ch. 9
F14	Retrofit catalogue figures carry no citations	Credibility	Medium	Closed 1 Sep; now a blocking gate

2.3 The three critical findings in detail

2.3.1 F1 — maximising a quantity that cannot be summed

The proposal defined a life-cycle-adjusted score

$$S_{b,i} = \frac{(E_{b,i} \times L_i \times f) - C_i^{\text{emb}}}{K_i} \quad [\text{kgCO}_2\text{e per thousand EGP}]$$

and maximised $\sum_{b,i} S_{b,i} x_{b,i}$ subject to a budget. The objection is arithmetic rather than stylistic: the sum of ratios with different denominators has no physical interpretation, so the reported objective value carries no unit and the programme is degenerate — it rewards many cheap high-ratio items regardless of the absolute benefit delivered.

The repair maximises *absolute* net life-cycle benefit and keeps the density strictly as a ranking key. In the shipped code that separation is explicit and commented in both places: `Candidate.score_per_kegp` carries the density for display and for the greedy heuristic, and `optimize/objective.py` exposes `density()` used by the heuristics only, while `solve_ilp` maximises the objective value itself.

Why the density was kept at all.

Deleting it would have been the tidier change and the wrong one. A density is the right sort key for a greedy fill, it is what a reader wants to see beside an option on a screen, and the two uses are legitimate. What was wrong was summing it under a constraint. Keeping the quantity and moving it out of the objective preserves the useful half of the proposal's idea.

2.3.2 F2 — a ranking reversal that was an accounting artefact

This is the finding with the largest consequence for the written argument. The proposal's Table 1 and Figure 3 presented a ranking reversal — solar leads on raw savings per pound, insulation

overtakes it once embodied carbon and service life are counted — as the central justification for life-cycle scoring. The review recomputed the proposal’s own four interventions on a consistent basis and found that the reversal is produced by crediting each measure over its *own* service life. Under a common 30-year horizon, and under equivalent-annual normalisation, the life-cycle ordering is identical to the raw kWh-per-EGP ordering.

The arithmetic behind that is worth stating because it is checkable in five minutes: solar’s embodied carbon of 95 000 kgCO₂e is roughly four per cent of its 2.43 million kgCO₂e of lifetime avoided emissions at 0.45 kgCO₂e/kWh. Embodied carbon at that scale cannot reverse a ranking.

The fix had two halves. The mechanical half — a fixed horizon $H = 30$, replacement embodied carbon charged, replacement *capital* not charged to the current budget — is `domain/lifecycle.py`. The narrative half replaced the claim with a stronger one that survives measurement: *the best measure is building-specific, so no portfolio-wide priority list is correct*. That claim is now **F7** in the harness and it passes; §6.5 reports it.

2.3.3 F3 — the optimiser’s principal input was unspecified

$E_{b,i}$, the annual energy saving of intervention i at building b , is the main input to the allocation. The proposal said it came “from Stages 1–2”, which are ingestion and forecasting — and those produce a single aggregate whole-building series. A whole-building meter cannot by itself say how much an HVAC replacement will save: that needs the share of consumption which is HVAC, and the fraction of that share the measure removes. Neither existed anywhere in the proposal.

The review’s verdict on this one is worth quoting for its diagnosis rather than its fix: the consequence is that *the coefficients will be invented during implementation*, and every downstream number inherits that. The fix specifies a three-layer decomposition and makes its coefficients an explicit, cited, version-controlled input owned by the environmental engineer — which is the origin of the two-file data contract described in Chapter 4 and of Equation (5.1) in Chapter 5.

2.4 The findings that changed the evaluation

2.4.1 F8 — the headline comparison was weaker than it looked

The review predicted that comparing an exact solver against a density-ordered greedy heuristic on an unconstrained knapsack would disappoint, because such heuristics land within one to three per cent of optimal. Presenting exact optimisation as the contribution and then reporting a two per cent improvement over a fifteen-line heuristic invites the obvious question.

Three repairs were prescribed, all of them honest, and all three shipped: headline the equal-split comparison, which is the status quo the proposal itself describes; report the greedy gap plainly; and construct the instance where greedy genuinely fails by adding a policy-style side constraint it cannot express. The district cap is that constraint, and Chapter 6 shows it doing the work.

The project later found that F8 was righter than the review knew.

The first implementation measured CP-SAT against plain greedy and reported a large advantage. The evaluation harness then caught the reason: plain greedy never revisits a funded building, so once every building holds its cheapest dense option it stops spending, and its benefit flatlines while the budget grows. The apparent advantage was measuring a baseline’s ceiling. A stronger baseline, `greedy_upgrade`, was written specifically so the comparison would be fair, and every headline in this report is quoted against it. That sequence — claim, harness contradicts claim, baseline repaired, claim narrowed — is the clearest single vindication of building the harness.

2.4.2 F9 — a statistic with a breakdown point of zero

The proposal flagged a reading when $|r - \mu| > k\sigma$ over a rolling window of residuals. Both μ and σ have a breakdown point of zero: the anomalies being detected are themselves inside the window, so they inflate σ , which raises the threshold, which hides the next anomaly. A sustained fault — precisely the “equipment left running” case the proposal names — progressively conceals itself, and the longer it runs the less likely it is to be caught.

Median and median absolute deviation have a 50% breakdown point, and the constant 1.4826 rescales MAD to be σ -consistent under normality so that k keeps its familiar reading. Chapter 7 reports what it took to make that detector reach its precision gate, which was not the threshold.

2.5 Scope, schedule and credibility

Three findings were not code at all.

F7 re-baselined the schedule and inverted the build order: the optimiser first, against CSV fixtures, with no infrastructure behind it, under the governing principle that *if the pipeline slips the team must still possess the artefact it is being judged on*. The cut list of \$1.5 was fixed at the same time.

F11 cut components that demonstrate nothing: a WireGuard tier for hardware that does not exist, an SMTP alerting service nobody reads during a presentation, and six independently deployed services collapsed into a modular monolith. The saving was assessed at two days, redirected into the evaluation harness and rehearsal.

F14 made `source_ref` a required column validated in continuous integration, so that an uncited row fails the build. It was the last known-open item in the harness and closed on 1 September; it is now a blocking gate rather than a documented exception, so a row that *loses* its citation fails the run.

2.6 The verification gates the review set

These were fixed before measurement, which is what makes them gates rather than descriptions. Table 2.2 lists them beside what the system now reports.

Table 2.2: The review’s verification gates, and the measured outcome. Sources: `out/evaluation/claims.csv` and the measured-data runs of Chapter 6.

Component	Metric	Pass condition	Measured
Optimiser	benefit vs equal split	$\geq 30\%$, OPTIMAL	+181% median; 120/120 OPTIMAL
Optimiser	benefit vs greedy	$\geq 0\%$, gap reported plainly	+1.28% median uncapped, reported plainly
Optimiser	solve time	< 500 ms at 50 buildings	slowest 197 ms, median 109 ms
Forecasting	MAPE vs seasonal naive	beats it, else auto-fallback	3.4% vs 5.8%; fallback fires and is reported
Anomaly	precision, recall	recall ≥ 0.8 at precision ≥ 0.6	precision 0.80 at recall 0.89
Integrity	tamper detection	modified <i>and</i> deleted row both fail	both detected; anchor comparison wired
Reliability	cold boot, no network	working demonstration < 5 min	seeded backfill; offline guarantee enforced by CSP

Every gate is met. The two that are worth reading twice are the solver gates, because they are met in opposite directions: the equal-split margin is very large and the greedy margin is very

small, and the project reports both rather than the flattering one.

Chapter 3

Architecture

3.1 One process, six modules, five containers

GEMP is a modular monolith with infrastructure containers around it. The module boundaries follow the logical architecture the proposal drew; the deployment is one application process plus four supporting services, orchestrated by Docker Compose on a single host.

That collapse is **F11** made concrete, and it was a judgement rather than a shortcut. Six independently deployed services means six Dockerfiles, six health checks and six failure modes for a workload whose heaviest steady-state operation is a batched `INSERT` every two seconds. The seams the review said would survive the collapse are package boundaries, not process boundaries — and the guarantee that keeps them honest is stated in code: `gemp.domain` imports no database, no broker and no web framework.

What that one guarantee buys.

Thirteen of the fourteen evaluation claims can be measured with no containers running at all — `python -m gemp.evaluate` loads the portfolio from GeoJSON, the catalogue from CSV and the parameters from YAML, expands the candidate set and solves. So the artefact the project is judged on runs when the pipeline is down, which was the entire point of building it first (**F7**). It is also why the optimiser could be developed before the database existed.

3.2 The five stages and the one loop

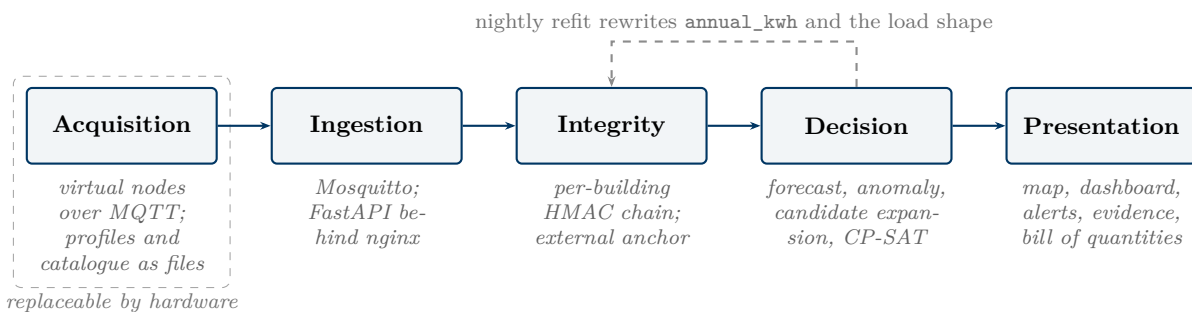


Figure 3.1: The five stages, and the one loop. Time series enter *only* through the MQTT interface, so a microcontroller with a clamp meter replaces the simulated nodes without touching the broker, the store or the engine. The dashed return is why the refit is scheduled inside the application process: it changes the values the candidate set is derived from, and in a separate process nothing would tell the API its cache had gone stale.

3.3 What crosses each boundary

Figure 3.2 is the deployment as it actually runs. Five containers, and the important detail is the fan-out below the ingest loop: it writes to three destinations at once, and the third is taken off the ingest thread on purpose.

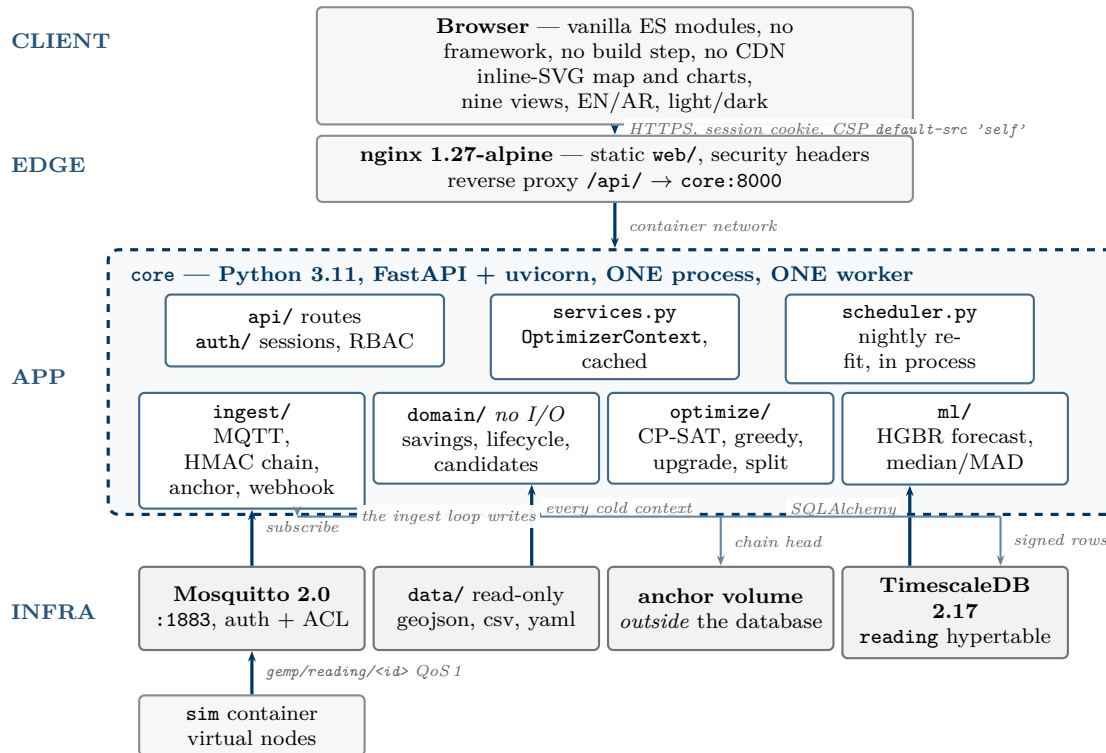


Figure 3.2: Every deployed component. **sim** publishes readings, **mosquitto** routes them, **core** signs and stores them and answers the API, **timescaledb** holds them, **nginx** terminates the edge and serves the front end. The fan-out below **ingest/** is the one place three destinations are written at once: the signed row into the database, the chain head into a volume the database cannot reach, and an optional notification outward — the last taken off the ingest thread so a slow endpoint can never stall ingestion. **domain/** performs no I/O at all.

3.4 Module inventory

3.5 Three decisions that look like details

3.5.1 The scheduler must be in-process

The nightly refit changes `building.annual_kwh`. The API caches an `OptimizerContext` holding the candidate set derived from those very values. If the two live in different processes, nothing tells the API its cache is stale and it keeps serving recommendations costed against yesterday's consumption until somebody restarts it. That is a silent wrong answer — the map still renders and the numbers still look plausible — which is the worst kind. In-process, the sequence is explicit and ordered:

```
refit → write annual_kwh → rebuild candidates → invalidate the cache
```

3.5.2 The application refuses to start under more than one worker

Two things in the process are per-process state and are correct only because there is one of them: the login throttle counters, and the ingester thread. Under `--workers 4` the throttle becomes

Table 3.1: The application’s modules, with what each owns. Line counts are from `git ls-files`; the repository holds 174 tracked files and roughly 42 300 lines across code, tests, front end and infrastructure.

Module	Lines	Owns
domain/	1 580	Physics and economics. No database, no broker, no web framework. <code>models</code> , <code>catalog</code> , <code>savings</code> , <code>lifecycle</code> , <code>candidates</code> , <code>portfolio</code>
optimize/	780	The core deliverable. <code>ilp</code> (CP-SAT), <code>baselines</code> (three), <code>prune</code> (dominance), <code>runner</code> (one interface), <code>objective</code> (four objectives)
ingest/	1 120	MQTT to signed storage: <code>consumer</code> , <code>integrity</code> , <code>anchor</code> , <code>live_anomaly</code> , <code>webhook</code>
ml/	1 290	<code>forecast</code> , <code>features</code> , <code>anomaly</code> , <code>dataset</code> , <code>evaluate</code> , <code>jobs</code>
api/	2 180	<code>main</code> (middleware, routes), <code>auth_routes</code> , <code>dashboard_routes</code> , <code>jobs</code> , <code>deps</code>
auth/	1 560	<code>service</code> (sessions, throttle, audit), <code>deps</code> (deny by default), <code>passwords</code> (scrypt), <code>roles</code> , <code>judge_pass</code> , <code>bootstrap</code>
evaluate/	1 320	The harness: <code>claims</code> , <code>sweep</code> , <code>measured</code> , <code>figures</code>
sim/	960	<code>node</code> (virtual sensors), <code>profiles</code> (load generation), <code>replay</code> (real UCI dataset)
web/	9 900	Nine screens as vanilla ES modules plus one stylesheet
tests/	6 600	548 collected cases across 27 files

four independent counters — five failures each, twenty attempts before anything is refused — and four ingesters race to write the same signed chain. Neither failure announces itself: the throttle still returns 429 eventually and the ingester still ingests.

So the assumption is checked rather than documented. `_refuse_multiple_workers()` raises at start-up on either spelling, the `--workers` flag or the `WEB_CONCURRENCY` environment variable, because the variable is the one that gets set in a compose file by somebody who never read the module.

3.5.3 No PostGIS, and Timescale objects outside Alembic

Footprints are GeoJSON in a JSONB column and the district is a plain string. The system never runs a spatial query — it groups by district for the cap constraint and hands geometry to the browser verbatim — so the extension would add a deployment dependency and buy nothing.

The hypertable and the hourly continuous aggregate live outside Alembic because a continuous aggregate cannot be created inside a transaction and Alembic wraps every migration in one. They are created by an idempotent bootstrap running with `AUTOCOMMIT`, which also lets the stack degrade to plain PostgreSQL on a machine where the container runtime is unavailable.

The chunk interval is a locking decision, not a storage one.

The interval is measured in *data* time, and data time runs at 720×. A one-day interval therefore produced a chunk every two minutes of uptime — about 1 400 of them across the replay this deployment accumulated. A query touching the whole hypertable takes a lock per chunk and per chunk index out of a shared pool sized `max_locks_per_transaction` × `max_connections`, and `SELECT max(ts) FROM reading` is the first thing every dashboard request runs. Past roughly a thousand chunks it fails with “out of shared memory”. The interval is now 30 days — about 144 000 rows per chunk — and the lock pool is raised to 1024 in the compose file. Both halves were needed.

Chapter 4

Data and provenance

4.1 The contract between two people

Two version-controlled files carry the entire interface between systems engineering and environmental engineering:

File	Owner	Holds
data/catalog.csv	Nada	cost model, saving fraction, service life, embodied carbon, applicability rule, citation
data/params.yaml	Nada	grid factor, horizon, end-use shares, condition multipliers, solar physics, anomaly threshold
data/buildings.geojson	versioned	the 50-building portfolio every published number was measured on

The rule is that **Nada edits values and never code; Ahmed edits code and never values**, and both files are schema-validated on every push so a malformed edit fails the build rather than silently producing wrong recommendations.

The contract was crossed once, deliberately, and it is recorded.

On 7 September — two days before submission — four values in `params.yaml` were changed from the code side against primary sources: the grid emission factor, the electricity tariff, the solar specific yield and the roof-orientation table. Each carries its source, its locator and the date it was fetched in a comment beside the value. Nothing was estimated: every saving fraction and cost in the catalogue, the condition multipliers, the end-use shares and the two solar cost terms were left exactly as they were, and the one value that could not be sourced was labelled a placeholder rather than given a citation that does not support it. The exception is documented in `data/README.md` rather than left for a reader to notice.

4.2 The portfolio, and two earlier versions that were wrong

The portfolio is fifty real, named public buildings in Greater Cairo, fetched from OpenStreetMap via Overpass and filtered to public amenities. Every building carries the name OSM holds for that way, its `osm_way_id`, and a district resolved by reverse-geocoding its own centroid through Nominatim. **No institution name is invented.**

The history of this file is the clearest example in the project of a defect that produced entirely plausible output.

The first version

asked for `way[building]` inside a bounding box, kept whatever was large enough, and labelled the results “Office Building 4”. An audit against OSM found what those fifty footprints actually were: twenty apartment blocks, seven residential, four commercial, three places of worship, a train station and a house. The building labelled “Admin 24X7 Building 23” was a mosque; “Office Building 11” was a shopping mall; “School Building 15” was a car-parts shop.

The second version

asked for public amenity and office tags, which fixed *what* the buildings were — but over New Cairo alone OSM names only twenty-six public ways in the size band, so nineteen of the fifty were still called “School (2)”.

The shipped version

asks over Greater Cairo, where OSM holds 135 named public ways in the same band, and **keeps only named ones**.

Why a plausible name is worse than no name.

A plausible ministry name on a footprint that is not that ministry is a fabricated record, and this project’s entire argument is the opposite claim. Keeping only ways OSM already names means a reader who doubts the portfolio can check any building in a browser by its way id. The cost is that half the names are in Arabic — the fixture is written with `ensure_ascii=False` so it stays reviewable.

Three selection rules do real work. *Group quotas* (state 20, education 12, health 8, safety 6, civic life 4) stop the portfolio filling with ministries and police stations, because a real programme covers schools and hospitals too and the optimiser has more to say when load shapes differ. *Spread* caps three buildings per 0.003° cell: without it, twenty-two consecutive unnamed `building=public` ways within a hundred metres took forty-four per cent of the slots — one site mapped as many structures, which would make the district cap meaningless. *Name-hint exclusions* drop international schools, academies and syndicates, and the limitation is stated rather than hidden: the filter is blunt in both directions.

4.2.1 What the portfolio looks like

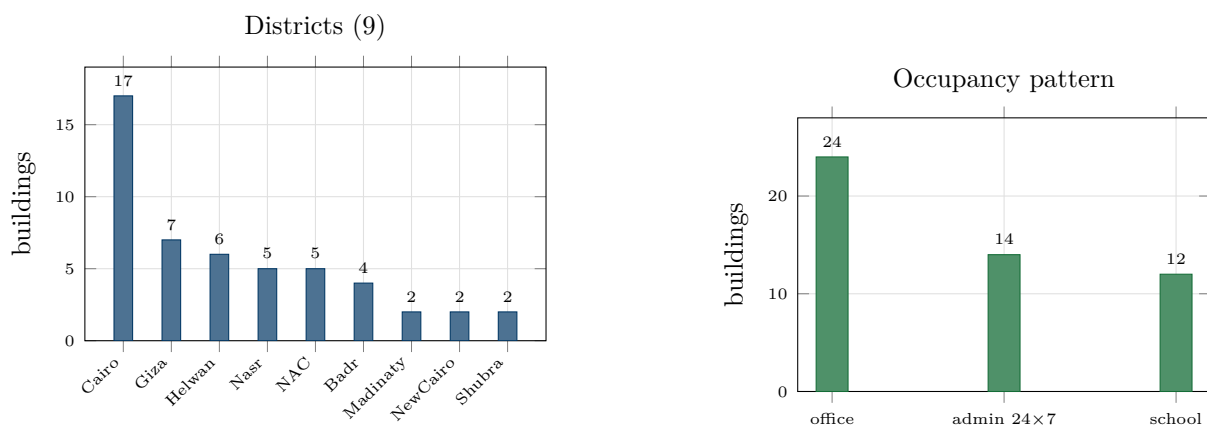


Figure 4.1: Portfolio composition, from `data/buildings.geojson`. Districts are the administrative units Nominatim resolves each centroid to, which is occasionally counter-intuitive: a building a Cairene would place in Maadi is recorded under Helwan, the boundary it actually falls inside. Occupancy is read off the OSM amenity tag rather than drawn at random — a hospital carries a 24×7 load, a school does not, and that mapping is most of why their retrofits are worth different amounts.

Table 4.1: Portfolio statistics, measured from the fixture. Operational attributes (occupancy, HVAC age, insulation quality, consumption) are synthetic and declared as such: no open dataset provides them for Egyptian public buildings, and a real deployment would collect them through a profile-entry form.

Attribute	Minimum	Median	Maximum
Annual consumption (kWh)	267 960	1 650 500	11 958 270
Floor area (m ²)	2 794	12 579	49 094
Roof area (m ²)	1 262	3 177	7 973
Glazing area (m ²)	380	2 524	8 828
HVAC age (years)	2	16	25
<i>Totals and mixes</i>			
Buildings	50, all Polygon footprints, all named by OSM		
Portfolio consumption	132.7 GWh/yr		
Insulation quality	poor 22, fair 14, good 14		
Roof orientation	flat 44, south 4, south-east 2		
HVAC type	package 21, split 16, chiller 13		

Heterogeneity is a requirement, not an accident.

The distributions are chosen so the portfolio varies in the three dimensions that make the best measure building-specific rather than universal: roof area relative to consumption, absolute roof area, and insulation quality. A portfolio of identical buildings would make the optimiser’s central claim untestable — every building would want the same measure, and a fixed priority list would be right.

4.3 The intervention catalogue

Six interventions, each with a saving fraction expressed against its *target end use* rather than against whole-building consumption.

Table 4.2: The catalogue. Every row carries a citation; an uncited row fails `python -m gemp.domain.catalog --validate --strict`, which gates submission.

Intervention	End use	Cost	Saving	Life	Source
Roof/envelope insula- tion	hvac	850 /m ² roof	0.18	30 yr	HBRC Journal 22(1) art.41 p.11; Egyptian Energy Building Code 2022
Double glazing retrofit	hvac	2 400 /m ² glazing	0.08	25 yr	ASHRAE Fundamen- tals 2021 ch.15; <i>J.</i> <i>Build. Eng.</i> 35:102042
High-efficiency HVAC plant	hvac	400 /m ² floor	0.30	15 yr	IEA, <i>The Future of Cooling</i> (2018)
BMS zone control	hvac	65 /m ² floor	0.12	10 yr	PNNL-25985 (2017, rev. Oct 2017)
LED lighting upgrade	lighting	83 /m ² floor	0.55	12 yr	Elsewedy Lighting price list, Sep 2024
Rooftop photovoltaics	generation	fixed + per kWp	—	20 yr	Fraunhofer ISE <i>Photo- voltaics Report</i> (2024) p.37

Two properties of the catalogue schema matter more than the values. `exclusive_group` keeps the rule that two HVAC plant replacements are not a bundle in the *data* rather than hard-coded in the solver. And `applies_if` is a deliberately small domain-specific language (`hvac_age_yr>=8`,

`insulation_quality in poor|fair`) rather than JSON, because the file is edited in a spreadsheet and JSON inside a CSV cell needs quote-doubling that Excel mangles.

4.4 Parameters, and where each one comes from

Table 4.3: Parameters on which published results depend. All sources retrieved 7 September 2026. Each reference names a page, chapter, article or query rather than a document.

Parameter	Value	Source and locator
Grid emission factor	0.3803 kgCO ₂ e/kWh	EEHC <i>Annual Report 2023/2024</i> p.30: environmental indicator for <i>thermal</i> plants reached 380.3 t CO ₂ /GWh
Analysis horizon	30 yr	Common-horizon requirement (F2)
Discount rate	0.0	Undiscounted base case; sensitivity is a parameter change
Electricity tariff	2.74 EGP/kWh	EgyptERA schedule effective April 2026, low voltage (380 V), “Other Users”: 274.0 Pt/kWh
Solar specific yield	1845 kWh/kWp	PVGIS v5.3, PVGIS-SARAH3 (2005–2023), at the portfolio centroid 30.0532°N 31.3813°E, optimal angles 28°/3°, system loss 10%
Orientation factors	measured	Same PVGIS query per aspect; Table 4.4
Soiling loss	0.06	Charged separately from the PVGIS system loss, so dust is not counted twice
Solar cost per kWp	22000 EGP	<i>Market estimate inside a stated bracket</i> , not a measurement
End-use shares	per occupancy	RCEEE Egypt Energy Outlook and NEEAP II: HVAC 50–60 % of public-building consumption
Condition multipliers	see text	<i>Declared engineering assumptions</i> , not a citation
Hourly marginal profile	placeholder	Not sourced. No public 24-hour marginal series exists for the Egyptian grid

4.4.1 The grid factor is an operating margin, not a grid average

This deserves a paragraph because the wrong reading of the same number is 12% away. The cited report states the indicator for *thermal* generating plants; the same page puts new and renewable generation at 11.65% of the total, from which a grid *average* of roughly 0.336 follows. GEMP credits *avoided* consumption, and the plant that stops burning when a building saves a kWh is a thermal one in essentially every hour of the Egyptian day. An operating margin is therefore the correct concept and the grid average is not.

Two candidates were considered and rejected, and the rejections are recorded so the choice can be checked rather than believed: the derived grid average, because it answers a different question and its arithmetic quietly treats share-of-generation as share-of-delivery; and a third-party whole-grid average of 0.563, because it sits 48% above the primary national source’s own thermal figure.

One conservatism is stated rather than modelled: a kWh saved *at the meter* avoids more than a kWh at the busbar, because network losses go with it. This is a generation-side figure, so every carbon saving reported here is understated by roughly the transmission and distribution loss rate.

4.4.2 Solar yield and orientation, measured rather than assumed

The table this replaced was symmetric — east and west both 0.88, north-east and north-west both 0.78 — and Cairo is not: west outperforms east by 1.8% because afternoons are clearer than mornings, and the guesses were optimistic by up to 11% on the northern aspects, which is

Table 4.4: Roof orientation factors, measured at 28° slope and normalised to due south. PVGIS azimuth convention: 0° south, negative east, positive west.

Aspect	Azimuth	Yield (kWh/kWp)	Factor
South	0°	1 845.02	1.000
South-west	+45°	1 773.48	0.961
South-east	−45°	1 755.01	0.951
West	+90°	1 581.15	0.857
East	−90°	1 552.53	0.841
North-west	+135°	1 337.99	0.725
North-east	−135°	1 310.13	0.710
North	180°	1 193.69	0.647
Horizontal	—	1 674.17	0.907

exactly where a solar option is closest to not being worth funding. The old yield of 1 650 carried no coordinates and no date and understated the resource by 11.8%.

A flat roof is modelled as mounted *horizontally* at the measured 0.907. Tilting to 28° would reach 1.000, but that needs row spacing to avoid self-shading and `usable_roof_frac` does not model spacing. The horizontal figure is the conservative of the two and it is measured rather than argued.

4.4.3 The placeholder, and why the negative result is the finding

The time-of-use marginal emission profile is twenty-four illustrative values and the report says so. What was searched, so nobody repeats the afternoon:

- **EEHC Annual Report 2023/2024, p.15** publishes the only public Egyptian 24-hour series — but it is *load* in megawatts, not emissions; the report’s own caption states the curve does not represent a typical 24 hours because of the load-shedding plan then in force; and it disagrees with the shipped array anyway, troughing at hour 7–8 and peaking at 20–21 where the array troughs at hour 2 and peaks at 18.
- **IRENA, *Renewable Energy Outlook: Egypt* (2018)**, all 121 pages: no hourly series, no load curve, no emission factor.
- **UNFCCC CDM tool AM-TOOL-07** gives the *method* for operating and build margins, annual rather than hourly; no published Egyptian result was found.
- **IGES grid emission factors**: annual OM/BM/CM by country, not hourly.
- **Electricity Maps** publishes hourly intensity for 200 zones, but it is an *average attributional* figure rather than a marginal one.

So the time-of-use layer is claimed as a **capability**: the optimiser consumes an hourly marginal profile wherever a system operator publishes one, and ships a labelled placeholder where none exists. Deleting the block entirely is a supported configuration — every consumer falls back to the flat factor — and is the right choice for a deployment that would rather claim nothing than claim this.

One invariant makes the comparison mean anything.

The 24 values are scaled so their mean is exactly the flat factor. A building with no measured shape is then costed identically under either objective — to 0.002%, the residue of rounding to three decimals — so any divergence between the two is attributable to a measured shape rather than to a change of units. A profile rescaled without that care would move every shapeless building silently. When the grid factor was cited on 7 September the array was rescaled by the same ratio, 0.84511.

4.4.4 Condition multipliers are declared assumptions

Saving fractions are adjusted by building condition: a poorly insulated building has more to gain from insulation. These multipliers are informed by the logic of the Egyptian Building Code — the gap between current performance and what the code requires — but no single table in the code lists them, so no clause can be pointed at. They are therefore declared as reasoned engineering assumptions based on diminishing returns and thermodynamic heat gain. Every other parameter in the file is sourced; this block is the declared exception, which is a stronger position than a vague citation would have been.

4.5 Validation, as a gate rather than a convention

`domain/catalog.py` is the enforcement point. It rejects a catalogue with duplicate ids, an `adj_key` with no matching block in the parameters, an end use missing from the share table, or an orientation missing from the solar table. End-use shares must sum to 1.0 per occupancy class to within 10^{-6} , because they partition a building's consumption.

One check exists because of a specific near-miss:

```
stray = sorted(k for k in raw if str(k).lower().startswith("tou") and k != "tou")
if stray:
    raise ValueError(
        f"params.yaml has key(s) {stray} resembling 'tou' but no 'tou:' block; "
        f"the TOU grid profile must live under 'tou: profile: [...]'"
    )
```

Pydantic ignores unknown keys rather than rejecting them, so a typo — a top-level `tou_profile:`, or `profile` misspelled inside the block — would validate clean and silently fall back to the flat factor. The entire point of the time-of-use layer is that it changes funding decisions; silently not having one is the failure this check exists to catch.

Chapter 5

The decision model

This chapter develops the model as equations, in the order the quantities are computed: from one metered number per building to a funded set. Each step names the review finding it answers.

5.1 From a meter reading to a per-intervention saving (F3)

The platform observes one number per building: total metered consumption. For building b and intervention i acting on end use $u(i)$,

$$E_{b,i} = A_b \cdot s_{o(b),u(i)} \cdot f_i \cdot \alpha_i(b), \quad (5.1)$$

where A_b is annual consumption, $s_{o,u}$ the share of end use u for occupancy class o , f_i the intervention's saving fraction *of that end use*, and $\alpha_i(b)$ the condition multiplier.

Expressing f_i against the end use rather than against total consumption is what keeps the catalogue comparable across buildings with different occupancy patterns. If the fraction were against total consumption, the numbers would stop being comparable between an office and a 24×7 facility, and the share table would have nothing to do.

A_b deserves a note. In Phase 0 it came from the fixture; from Phase 2 it is the annualised forecast, and `ml/forecast.py` calls that function “*the single point at which the metered time series reaches the allocation decision.*” Until the nightly refit runs, the optimiser is costing against the figure the portfolio shipped with. The database records which is in force per building (`annual_kwh_source`), because that distinction matters when explaining a recommendation.

5.1.1 Generation takes a separate path

Rooftop photovoltaics adds supply rather than removing demand, so it does not fit Equation (5.1):

$$P_b = \frac{R_b \cdot \rho}{m} \quad [\text{kWp}] \quad (5.2)$$

$$G_b = P_b \cdot Y \cdot \omega(b) \cdot (1 - \lambda) \quad [\text{kWh/yr}] \quad (5.3)$$

$$E_{b,\text{solar}} = \min(G_b, A_b \cdot \kappa) \quad [\text{kWh/yr}] \quad (5.4)$$

with R_b the roof area, ρ the usable fraction, m the m^2 per kWp, Y the specific yield, ω the orientation factor, λ the soiling loss and κ the self-consumption cap. The capital cost is *not* proportional:

$$K_{b,\text{solar}} = K^{\text{fixed}} + k \cdot P_b. \quad (5.5)$$

Equations (5.4) and (5.5) are what make the ranking flip between buildings.

Under a purely proportional cost model the benefit density is scale-invariant and can never flip: every building would rank solar identically against fabric measures. The fixed term — inverter, mounting, grid connection, permits — makes small installations markedly worse per unit benefit. The self-consumption cap does the opposite asymmetry: with no export tariff, generation beyond on-site demand delivers no saving, so a large roof on a low-consumption building clips its benefit while its cost does not clip. Both were absent from the proposal, and both are mechanisms the review identified as physically real and already representable.

5.2 A common horizon, and embodied carbon (F2)

Measures whose service lives differ by a factor of three cannot be compared over their own lifetimes without the comparison encoding the lifetimes. Every measure is credited over a common horizon $H = 30$ years and charged for the replacement cycles that horizon implies:

$$V_{b,B} = \underbrace{E_{b,B} \cdot a(H, r) \cdot \gamma}_{\text{avoided operational carbon}} - \underbrace{\sum_{i \in B} c_i(b) \cdot n_i(H)}_{\text{embodied carbon over the horizon}}, \quad (5.6)$$

where $a(H, r)$ is the annuity factor at discount rate r — which reduces to H when $r = 0$, keeping the output reconcilable with the proposal's undiscounted figures — γ the grid emission factor, $c_i(b)$ the embodied carbon of one installation, and $n_i(H) = H/L_i$ the number of replacement cycles.

Two details in that equation are decisions rather than algebra.

$n_i(H)$ is **fractional on purpose**. A 12-year lamp inside a 30-year horizon is charged 2.5 embodied hits, not 3. Rounding up would penalise short-life measures at horizon boundaries for no physical reason.

Replacement **capital is not charged** to the current budget, only replacement embodied carbon. A ministry allocating this year's money pays for one installation now; the lamp replacement in year 12 comes out of a future budget. Mixing the two would put a cost in the constraint that nobody is being asked to fund today.

5.3 Bundling composes multiplicatively within an end use (F4)

Measures acting on the same end use do not add. Insulation reduces cooling load; an HVAC replacement raises coefficient of performance; both act on $E_{\text{hvac}} = \text{Load}/\text{COP}$. Adding their fractions overstates the combined saving by 20–40%. Writing B_u for the members of bundle B acting on end use u , and $A_{b,u} = A_b s_{o(b),u}$:

$$E_{b,B} = \sum_u A_{b,u} \left[1 - \prod_{i \in B_u} (1 - f_i \alpha_i(b)) \right]. \quad (5.7)$$

Multiplicative *within* an end use and additive *across* end uses is the only combination right in both directions: two measures on the same load interfere, two measures on different loads do not. For a bundle of one this reduces to Equation (5.1), which is stated first because it is the case a reader checks by hand.

A whole-building ceiling caps the total at 95% of consumption, and it is applied *pro rata* across the end-use slices rather than by clipping the sum. Scaling every slice by the same factor preserves the ratios the composition produced — which matters because those slices are what the time-of-use weighting later reads.

Pre-expansion is what keeps the solver clean.

Every compatible combination is expanded into its own candidate option, with costs added and savings composed. The payoff is structural: with combinations pre-expanded the solver needs *no* interaction constraints at all, because the existing at-most-one-per-building constraint already forbids double counting. The model stays a clean multiple-choice knapsack, stays linear, and stays explainable. The interaction physics lives in the catalogue — the environmental engineer’s domain — rather than in the solver, which is the systems engineer’s. That is a clean ownership boundary as well as a modelling one.

A guard rail sits in the expansion path rather than only in a test:

```
def assert_physically_possible(building, annual_kwh_saving, params, context=""):
    ceiling = building.annual_kwh * params.guards.max_building_saving_frac
    if annual_kwh_saving > ceiling + 1e-6:
        raise SavingsError(...)
```

A recommendation that saves 130% of a building’s consumption is the single most embarrassing thing this system could put on a screen during judging, and it is the natural failure mode of a mis-entered saving fraction or a duplicated bundle.

5.4 The allocation problem (F1, F8)

Let C_b be the candidate options at building b — single interventions and compatible bundles — with cost k_c and benefit v_c . With binary x_c :

$$\text{maximise} \quad \sum_b \sum_{c \in C_b} v_c x_c \quad (5.8)$$

$$\text{subject to} \quad \sum_{c \in C_b} x_c \leq 1 \quad \forall b \quad (5.9)$$

$$\sum_b \sum_{c \in C_b} k_c x_c \leq K \quad (5.10)$$

$$\sum_{b \in D} \sum_{c \in C_b} x_c \leq m \quad \forall \text{ districts } D \quad (5.11)$$

$$x_c \in \{0, 1\}.$$

Equation (5.9) makes this a multiple-choice knapsack. Equation (5.10) is the budget, in *upfront capital only*. Equation (5.11) is the policy constraint that motivates exact optimisation, and it is the one a greedy rule cannot express at all.

The objective is **absolute** benefit. Densities with different denominators do not sum, so maximising a sum of densities optimises nothing meaningful; density survives only as the sort key of the greedy baselines and as a display quantity.

Four objectives are exposed and solved identically, because a climate target and a budget holder do not always rank the same measure first and hiding that behind one number would be a worse tool rather than a simpler one:

Objective	Maximises	Unit
lca_carbon	net life-cycle carbon at the flat factor	kgCO ₂ e
tou_carbon	the same, weighted by the building’s measured shape	kgCO ₂ e
raw_kwh	first-year energy saved	kWh/yr
egp_saved	first-year money saved at the tariff	EGP/yr

5.4.1 Integrality, and why rounding is safe here

CP-SAT is an integer solver. Costs are already whole EGP; benefits are rounded to whole units. At portfolio magnitudes — benefits of 10^5 to 10^7 — unit rounding is far below the uncertainty in the catalogue’s own coefficients, which is the honest justification rather than an appeal to convenience.

5.4.2 Dominance pruning

An option dominated at its own building — costing no more and delivering no less — can never appear in an optimal solution, because replacing it with its dominator is feasible and no worse. Pruning to the per-building cost/benefit Pareto frontier removes 62% of the candidate set ($1\,448 \rightarrow 551$) before solving.

The argument has to hold for every solver, and it is written down for each:

- **CP-SAT** — the budget constraint only loosens, the objective only improves, and the multiple-choice and district-cap constraints are untouched because it is still one option at the same building in the same district.
- **Greedy** — if A dominates B then A ’s density is at least B ’s, so greedy reached A first and skipped B as a used building.
- **Equal split** — A costs no more, so it is affordable whenever B is, and scores at least as well; the per-building maximum is unchanged.

Pruning depends on which objective is active — an option can be dominated on carbon but not on raw kWh — so it happens at solve time rather than at expansion time. `test_pruning_does_not_change_any_solver_result` checks the invariance across every solver $\times$ objective $\times$ budget combination, and claim **F1-b** re-measures it.

5.5 Time-of-use marginal weighting

Under a single constant emission factor the metered time series changes nothing about which building is funded: it sets one scalar. The sharpest question a reviewer can ask of an instrumented platform is whether the readings it ingests change the decision it produces, and until this layer the honest answer was *no*.

A marginal profile g_h , $h = 0 \dots 23$, states the carbon of the next generating unit dispatched in hour h . The nightly refit derives, alongside each building’s mean load, a normalised hour-of-day shape $\sigma_{b,h}$ from that building’s own metered history. The effective factor for its saved energy is

$$\gamma_b = \frac{\sum_h \sigma_{b,h} g_h}{\sum_h \sigma_{b,h}}, \quad (5.12)$$

which replaces the constant γ in Equation (5.6) for the `tou_carbon` objective. An office that empties at 15:00 saves most of its kWh before the evening peak; a 24×7 clinic keeps burning through it. Same national average, genuinely different carbon per saved kWh.

The flat path is retained unchanged rather than redefined, and the weighted benefit is carried *alongside* the flat one on every candidate. That is what lets the harness measure old against new instead of silently replacing the baseline, and it is why `lifetime_benefit_kgco2e` and `lifetime_tou_benefit_kgco2e` coexist on the same object. With no profile loaded the two are equal by construction, so every constructor written before the feature — including every test fixture — keeps working.

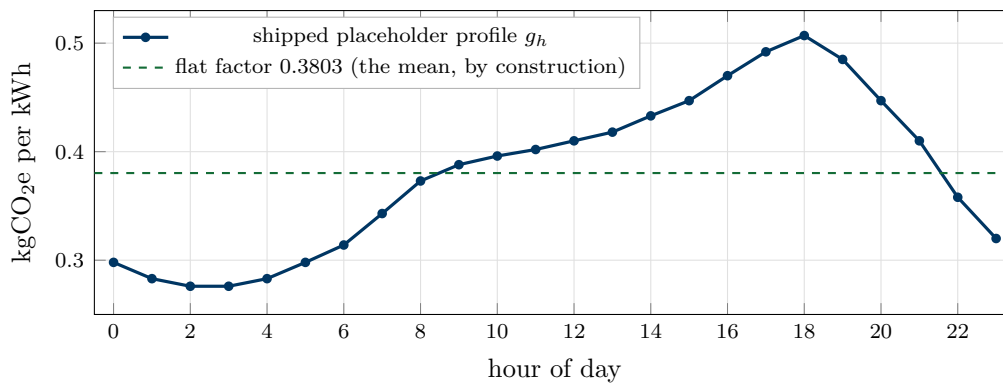


Figure 5.1: The 24 hourly factors, scaled so their mean is exactly the flat factor — the invariant that makes a building with no measured shape cost the same under either objective. The array encodes a 02:00 trough at 0.276 rising monotonically to an 18:00 gas peak at 0.507, a $1.84\times$ trough-to-peak ratio. There is deliberately *no* midday solar dip: an earlier draft of the comment beside this array claimed one and the array never had it, which is the kind of mismatch a reader checks. A real profile for a grid with meaningful daytime photovoltaics probably should dip around 11:00–14:00, and adding it would move the result — that is a measurement to source, not a shape to draw.

Where the weighting is applied, and the honest limit of it.

`effective_grid_factor` is called once per *building*, not once per end use, even though the per-end-use breakdown is passed in. That is the fallback depth from the plan: buildings re-rank against each other, so the funded set moves, but measures *within* a building do not re-rank, so solar cannot flip on this path. The seam for the deeper version is in place and documented rather than claimed as done.

Chapter 6

Results

Everything in this chapter is produced by `python -m gemp.evaluate` against the parameters of Table 4.3 on the fifty-building portfolio. The solver sweep is 480 runs: 20 budgets from 10 to 200 million EGP, $\times 3$ objectives, $\times 2$ cap settings, $\times 4$ solvers. Figures are drawn from the same rows the claims were measured on, so a figure cannot show a run no claim was checked against.

6.1 Allocation quality at one budget

Table 6.1: The four solvers at a budget of 50,000,000 EGP, objective `lca_carbon`. Transcribed from `out/evaluation/optimizer_sweep.csv`.

Solver	Funded	Spent (EGP)	Life-cycle kgCO ₂ e	vs equal split
<i>Unconstrained</i>				
<code>equal_split</code>	29	21 940 122	39 586 969	baseline
<code>greedy</code>	42	49 995 739	135 962 422	+243%
<code>greedy_upgrade</code>	42	49 995 739	135 962 422	+243%
<code>cpsat</code>	15	49 969 015	170 238 791	+330%
<i>At most two funded buildings per district</i>				
<code>equal_split</code>	15	11 274 593	22 952 097	baseline
<code>greedy</code>	18	22 877 215	75 835 137	+230%
<code>greedy_upgrade</code>	18	49 972 148	159 744 320	+596%
<code>cpsat</code>	10	49 988 795	161 511 232	+604%

The first panel contains the result worth pausing on. CP-SAT funds **fifteen** buildings where the heuristics fund **forty-two**, and delivers a quarter more life-cycle carbon for the same money. It is buying deep retrofits of a few high-consumption buildings rather than the cheapest option on every building it can reach — which is precisely the behaviour a density-ordered rule cannot produce, because a density-ordered rule takes the cheap dense option first and then has no way to go back.

The second panel shows the constraint doing the work. Plain greedy commits its district slots to cheap dense options and then cannot spend the remainder: it places only 22.9 of the 50 million.

6.2 What the budget buys, across the whole range

6.3 Where exact optimisation earns its place

Over the twenty `lca_carbon` budgets the medians are +1.28% **unconstrained** and +8.63% **capped**, a ratio of 6.7 $\times$.

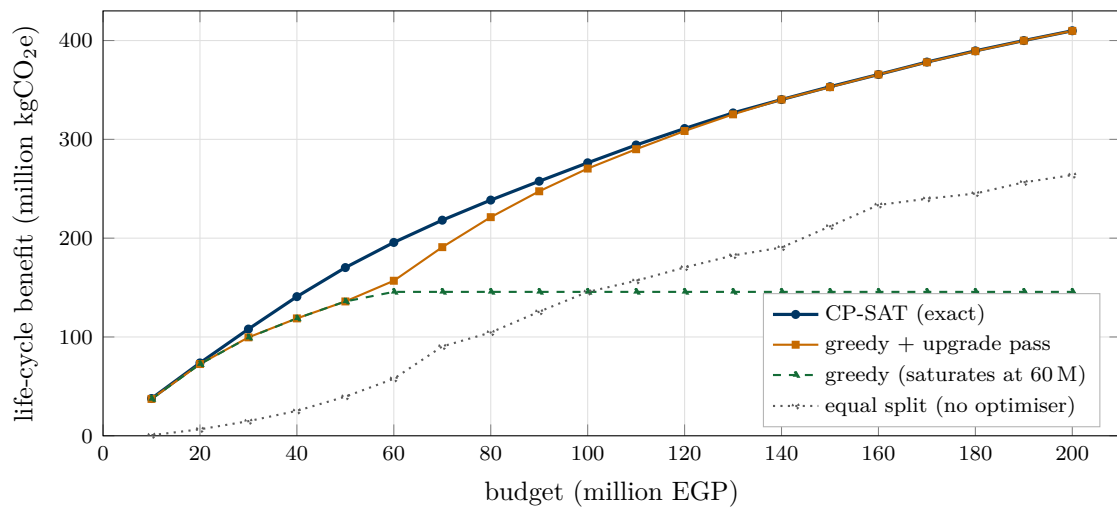


Figure 6.1: Life-cycle benefit against budget, objective `lca_carbon`, unconstrained. The flat green line is claim **F8-f** made visible: plain greedy never revisits a funded building, so once every building holds its cheapest dense option the algorithm cannot place another funded pound. Its benefit is pinned at 145.6 M kgCO₂e from 60 M EGP onwards, it leaves up to 72% of the budget unspent, and above roughly 100 M even equal split overtakes it. Quoting an advantage against *that* baseline measures its ceiling rather than the solver’s value, which is why every headline in this report is quoted against the upgrade pass.

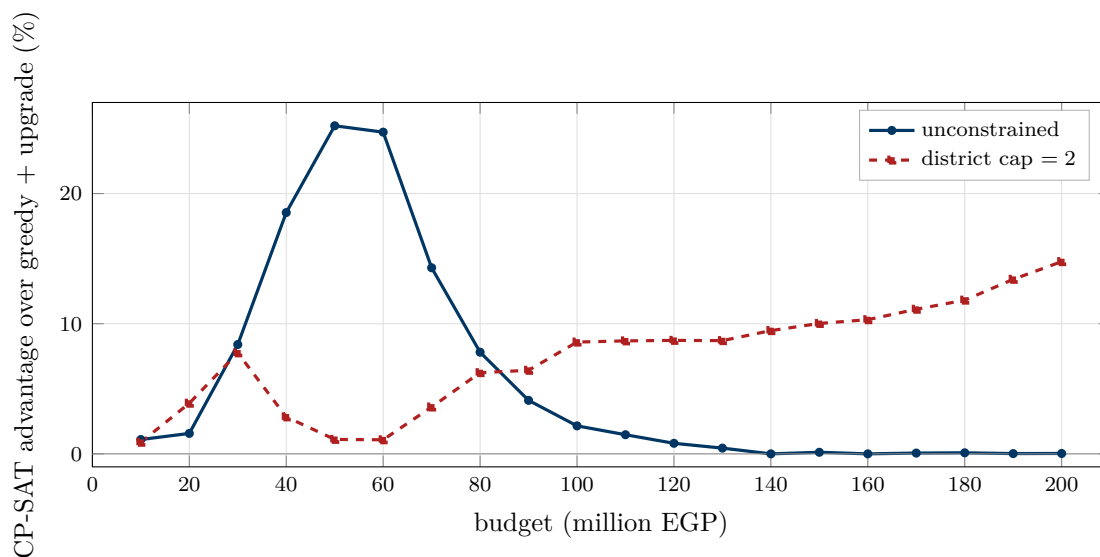


Figure 6.2: The two shapes a median conceals. The unconstrained advantage is a hump: it peaks at 25.21% at 50 M EGP — where the budget binds hardest and the choice of which building to skip decides the answer — then decays to 0.03% by 200 M once there is enough money to fund everything worth funding. The capped advantage does the opposite and *grows* to 14.77%, because the cap keeps binding after the budget has stopped.

Two medians that differ, and why both are right.

The harness reports +1.25% and +8.1% for a ratio of $6.5\times$, where the aggregation above gives +1.28%, +8.63% and $6.7\times$. The difference is scope, not disagreement: claim **F8-a** takes its median over *sixty* budget instances — twenty budgets across all three objectives — while the figures quoted here are the twenty `lca_carbon` budgets the curves are drawn from. Both are stated rather than reconciled into one, because silently picking whichever is more flattering is the habit this project exists to avoid.

This is why the project’s own earlier headline — an apparent advantage of 10% unconstrained and 134% under a cap — **must not be quoted**. It was measured against plain greedy, and it was measuring a baseline’s ceiling. The defensible claim is the narrower one: exact optimisation earns its place through side constraints no greedy variant can express, and through returning a proven optimum, not through a large unconstrained margin.

6.3.1 Solver performance

All 120 CP-SAT runs across the sweep returned status `OPTIMAL`: none fell back to a feasible-but-unproven solution and none hit the time limit. Solve time was a minimum of 61 ms, a median of 109 ms and a maximum of 197 ms, against the review’s gate of 500 ms at fifty buildings.

The slowest solve is a property of the machine, not of the model.

The paper quotes 244 ms as the slowest solve and this report quotes 197 ms. Both are real measurements of the same code on different hardware. Any figure of this kind should be read as bounded by the gate rather than as a constant of the system, which is why the claim is written as “within its time limit” and not as a number.

A fallback exists and is labelled rather than hidden. If CP-SAT expires without a feasible solution, `runner.solve` runs the strong heuristic instead and returns status `FALLBACK_FROM_<status>`, so nobody can mistake a fallback for a proven optimum. On this portfolio it has never fired.

6.4 A negative result: the life-cycle layer changes almost no decisions

At **16 of 20** budgets the `lca_carbon` and `raw_kwh` objectives fund *the same buildings with the same interventions*. At the four where they differ, the carbon cost of having ranked by raw kWh is at most 0.01% of what a carbon-optimal allocation delivers. Embodied carbon is a median 9.5% of gross avoided emissions across the 1 448 candidate options, and the options that actually get funded are the ones where it is smallest. Not one candidate option in the portfolio is net-negative over the horizon.

This is a negative result for a component the project considered central, and it is reported as one. The layer’s refusal to fund a measure emitting more than it avoids remains correct behaviour and remains tested — by *constructing* such an option rather than by relying on the portfolio to contain one — but it can no longer be offered as an observed finding about this portfolio.

Citing one input moved the finding, and that is the methodological result.

The grid factor was an uncited 0.45 for most of the project. Replacing it with the sourced 0.3803 moved the embodied share from a median 8.1% to 9.5%, and the number of budgets at which the two objectives disagree from two to four. The mechanism is transparent: the operational term of Equation (5.6) scales with γ and the embodied term does not, so lowering γ raises the relative weight of embodied carbon. Forcing γ back to 0.45 reproduces the earlier figures exactly, which is what makes the movement attributable to the citation rather than to a drifting portfolio. The finding has not reversed — it has moved perhaps a fifth of the way towards reversing, on a change to a single input. Provenance is not clerical work downstream of the analysis; it is a determinant of it.

6.5 A second negative result: no portfolio-wide priority list is correct

The best single measure by benefit density is building-specific. Across the fifty buildings:

Densest single measure	Buildings where it wins
LED lighting upgrade	28
BMS zone control and scheduling	21
Roof/envelope insulation	1
Rooftop photovoltaics	0

Rooftop photovoltaics is *never* the densest single option at any of the fifty buildings. At the installed cost of Table 4.3, generation cannot compete per EGP with cheap efficiency — which is the ordinary efficiency-before-generation loading order, arrived at here by measurement rather than by assertion, and directly contradicting the narrative the proposal offered.

The practical consequence is the argument the platform exists to make: a portfolio-wide priority list — “insulate everything, then re-lamp” — is not right, because which measure wins depends on the building. A building whose HVAC plant is four years old favours lighting; the same building at ten years favours controls.

This claim was reworded before its number moved, deliberately.

The gate originally demanded `solar_wins == 0` with no tolerance, while backing a headline in the paper. Whether rooftop solar wins somewhere under a marginal grid profile is a *finding to report*, not a condition for the sentence to hold — so the claim was rewritten to measure what it always meant, the diversity of winners (`distinct > 1`). Rewording a gate after it fails is rationalisation; rewording it before, while stating why, is not.

6.6 The claims harness output

Three properties of the harness are deliberate and are what make the table worth anything.

Claims are gated and can fail. Each carries a threshold fixed in advance — “median gap $\leq 25\%$ ”, “worst carbon shortfall $\leq 1.0\%$ ”, “capped gap at least $3\times$ the uncapped”. A claim that cannot fail is a sentence, not a measurement.

Insufficient evidence reports skip, never pass. The database-backed claims skip when no refit has run rather than quietly reporting the last value they saw. A median over fewer than three instances skips, because a median over two instances is not a median.

Solvers are paired on identical instances, keyed by (budget, objective, cap) rather than by list position, so a changed grid cannot silently compare a capped run against an uncapped one.

Table 6.2: All fourteen claims, transcribed verbatim from `out/evaluation/claims.csv`. Gates were fixed before measurement. **F15** reports SKIP in the file-only run by design and PASS with a database attached (§6.7).

ID	Verdict	Claim	Measured
F1-a	PASS	CP-SAT is never worse than any baseline on an identical instance	360/360 instances, worst gap -0.00% (3 exact ties)
F1-b	PASS	Dominance pruning does not change the optimum	3/3 budgets identical
F1-c	PASS	Candidate expansion is deterministic	1 448 options, 0 differing on re-expansion
F4	PASS	Interventions sharing an end use compose multiplicatively	662/662 same-end-use bundles strictly sub-additive; 0 exceed the sum of their parts
F5	PASS	Optimising raw kWh instead of life-cycle carbon costs almost nothing	worst carbon shortfall 0.01% (gate $\leq 1.0\%$); identical set at 16/20 budgets
F7	PASS	No single measure wins portfolio-wide	3 distinct winners over 50 buildings
F8-a	PASS	Unconstrained, CP-SAT beats the strong heuristic only slightly	median $+1.25\%$ over 60 budgets (gate $\leq 25\%$)
F8-b	PASS	Under a district cap the gap becomes decisive	median $+8.1\%$ capped vs $+1.2\%$ uncapped, $6.5\times$ (gate $\geq 3\times$)
F8-c	PASS	CP-SAT returns a proven optimum within its time limit	120/120 OPTIMAL, slowest solve 197 ms
F8-d	PASS	Optimised allocation beats an equal-split budget policy	median $+181\%$ (CP-SAT), $+158\%$ (upgrade); 0 losses
F8-e	PASS	CP-SAT benefit is non-decreasing in budget	0 violations
F8-f	PASS	Plain greedy saturates and cannot spend a large budget	flat from 60 M EGP; up to 72% unspent; 15 of 20 budgets at the ceiling
F15	SKIP	The TOU-weighted objective changes which buildings get funded	fixture portfolio carries no measured load shapes
DATA	PASS	Every catalogue row carries a citation	6/6 rows cited

One claim exists to catch a modelling bug rather than to make a point.

F1-a asserts CP-SAT is never worse than any baseline. Every baseline explores a subset of what the exact solver can express, so a loss on an identical instance means they are not solving the same problem — a bug in the model, the constraints or the objective mapping. It also needed a tolerance: where two solvers fund the same buildings for the same money their totals still differ in the last bit or two because they sum the same terms in a different order, measured at a relative 5×10^{-16} . A strict comparison reported that as CP-SAT losing, failing a claim on a run where every allocation was correct. The tolerance is 10^{-9} — four million times above the noise and still finer than the smallest unit any objective is reported in.

6.7 Results that require the metered series

Five claims cannot be measured from files alone; they need a database holding readings and a completed refit. Provenance is given per result because these come from different runs, and combining them silently would be the failure this project argues against.

6.7.1 The metered series reaches the allocation decision (F15)

This is the claim the time-of-use layer exists to make, and the one that answers “does any of the data you ingest change what you fund?”. After a refit wrote an hour-of-day shape for **50 of 50** buildings, the TOU-weighted objective funded a *different set* from the flat-factor objective at **1 of 3** probe budgets (25, 50 and 100 million EGP), against a gate of at least one. The claim passes.

The size of that number deserves care rather than enthusiasm. One budget in three is a modest effect, and it rests on a profile labelled a placeholder: the *mechanism* is demonstrated

— a measured per-building shape, carried into the objective, changing an allocation — while the *magnitude* is a property of an illustrative profile and would move with a sourced one.

An independent check during the pre-production audit confirmed the channel is live with the expected sign and scale: an evening-peaked building costed +23.3% against the flat factor and a night-peaked one −22.0%, a 45-point spread. That spread is what lets shapes change funded sets at all.

6.7.2 Forecasting (F3)

On the deployed host holding twenty-four months of history, per-building forecasting reads a median held-out MAPE of **3.4%** against a seasonal-naive baseline’s **5.8%**, with the learned model retained at 49 of 50 buildings. On a shorter twelve-month history the same pipeline reads 7.8% against 17.3%, retained at 48 of 50.

Two things there are worth more than the headline. The fallback rule *fires* — one building on the longer history and two on the shorter keep the seasonal-naive predictor because the learned model did not beat it on held-out data. A rule that never fires is ceremony. And the learned model’s *relative* advantage is larger on the shorter history ($2.2\times$ against $1.7\times$), which is consistent with a seasonal-naive predictor needing a long clean history to be competitive.

6.7.3 Anomaly detection (F9)

Episode-level detection reads **precision 0.80 at recall 0.89** with $k = 5$, against review gates of 0.6 and 0.8. Both are met. Chapter 7 gives the threshold study and the two measurement artefacts that had to be fixed before any of these numbers meant anything.

6.7.4 Integrity (F5-b)

Chain verification over a sample reports 5/5 chains internally consistent, 5/5 heads matching the external hourly anchor, and 5/5 agreeing with the database’s own copy of the head. The three checks are separate on purpose: the first detects modification or reordering inside the chain, the second detects truncation of the tail, and the third detects a database whose stored head has been edited to match a forged chain.

This claim once failed, and the failure was the feature working.

On 27 August **F5-b** reported 0/5 chains verifying, with b001 flagged **deleted** at sequence 4715. It was real: that building held 471 798 rows across a sequence span of 473 637 — 1 839 missing, in exactly one contiguous gap. The gap mapped to the wall-clock window in which the **sim** container was crash-looping twenty-nine times on an anchor permission error, publishing readings and dying, consuming sequence numbers on batches that never committed. So the tamper-evidence layer detected real data loss from a real incident, which is the strongest evidence for it in the project. It cleared on a re-seed, as predicted.

Chapter 7

Forecasting and anomaly detection

7.1 Why there is a forecaster at all

The honest justification is narrow and worth stating before the mechanics. Forecasting feeds the optimiser at exactly one point: the annualised estimate A_b in Equation (5.1). `ml/forecast.py` says so in the docstring of the function that produces it — “*the single point at which the metered time series reaches the allocation decision*”. Everything else the forecaster does serves the anomaly detector, which needs an expectation to compute a residual against.

7.2 Model choice (F10)

The proposal named “scikit-learn / Prophet”. **F10** rejected Prophet on two grounds: a compiled probabilistic backend that materially inflates image size and build time, and weak performance on sub-daily building load where the dominant signal is hour-of-week. What shipped is `HistGradientBoostingRegressor` — already in scikit-learn, so no new dependency — trained per building, with a seasonal-naive predictor retained whenever it wins on held-out data.

The baseline is allowed to win, and that rule is kept even though it rarely fires.

A forecaster that loses to “the same hour last week” and ships anyway is a liability. On the current portfolio the learned model is retained at all fifty buildings, so the rule reads as ceremony — until the day a building’s history is too short or too erratic for it, which is exactly when a forecaster that shipped without the check would be believed. The rule costs one comparison and removes a whole class of silent failure.

7.3 Features, and the one that mattered most

Four families, in descending order of how much they explain building load: lags (the strongest single predictor of hour t is hour $t - 168$, because load is a weekly occupancy schedule); calendar, encoded cyclically so hour 23 sits next to hour 0; rolling statistics, which carry season and weather without a weather feed; and the hour-of-week profile with the lags expressed relative to it.

The Egyptian calendar is not a detail. The weekend is Friday and Saturday — a model assuming a Western week is wrong two days in seven — and public holidays and Ramadan shift the shape of the curve for a month. One definition of that calendar is imported by both the simulator and the feature builder, and when the two disagreed the model mispredicted holiday load by roughly seventy per cent while the detector faithfully reported every hour of it: the five worst days in the whole evaluation were public holidays, and the top twenty dates accounted for 58% of every false alarm.

Giving the model the calendar is not leakage.

Public holidays and Ramadan dates are published years ahead, and any real deployment would supply exactly this. A load forecaster that does not know when the building is closed is not a serious forecaster. What *would* be leakage is telling the model about the injected faults, which nothing does.

7.3.1 Predicting the ratio, not the load

This is the change that closed the anomaly precision gate, and it is a modelling insight rather than a tuning one. Instead of predicting load, the model predicts the *ratio* to a causal hour-of-week profile:

```
profile = train["profile"].clip(lower=MIN_PROFILE_KW)
model.fit(train[list(FEATURE_COLUMNS)], train["kw"] / profile)
```

A tree adds leaf values. In kilowatts, a public holiday is therefore a per-building, per-hour constant that must be learned from the handful of holidays in the training window. In ratio space the same fact is “ $0.3\times$ ” — one split that holds at every hour and every season — and the diurnal and weekly shape is carried by the profile instead of being rebuilt from lags that contradict it at every step change.

Measured on the seeded portfolio: held-out MAPE $3.83\% \rightarrow 3.24\%$, and episode precision at recall 0.875 moved $0.554 \rightarrow 0.827$.

The precision gain is far larger than the accuracy gain, and that asymmetry is the finding.

A 0.6-point MAPE improvement producing a 0.27 jump in precision only makes sense if what was removed was not noise but a *systematic* error concentrated somewhere. It was: relative residual dispersion measured 0.125 at hour 00 against 0.06 for the rest of the day, and 640 of 1 379 false alarms began at hour 00 on a Friday or a public holiday. Egyptian load steps at midnight into the weekend or a holiday while every lag feature reports the building was busy an hour ago. The detector scales residuals against a window pooled across all hours, so one systematically worse hour breaches the threshold on ordinary days.

The profile itself is strictly causal, which is the whole difficulty. Grouping by (day of week, hour) and taking a trailing median inside each group means each point is compared with the same hour of previous weeks and never with itself — `shift(1)` inside a group moves the window back one *week*. A median rather than a mean, because the history contains faults and a stuck meter in one of the four weeks must not drag the expectation it is about to be measured against.

Temperature is deliberately absent. It would help — cooling is the largest single load component — but the platform has no weather source, and inventing one would be a worse answer than saying so.

7.4 Anomaly detection (F9)

7.4.1 A robust statistic, and a relative residual

The rule is

$$|r_t - \text{med}(r)| > k \cdot 1.4826 \cdot \text{MAD}(r), \quad (7.1)$$

whose breakdown point is 50% against the mean/ σ form’s zero. The constant 1.4826 rescales MAD to be σ -consistent under normality, so k keeps its familiar reading and the proposal’s starting value carries over.

Two refinements are not in the review and both were forced by measurement.

The residual is relative, not absolute. Building load is strongly heteroscedastic: an office drawing 300 kW at noon and 40 kW at 3 am produces absolute residuals an order of magnitude apart for the same proportional error. A single pooled rolling scale is then far too tight for daytime and far too loose for night. Measured on the seeded portfolio, the absolute form flagged 7.3% of all hours at $k = 3$ — roughly twenty-five times what a Gaussian would give, almost all of it daytime noise. Dividing by the expected value makes the residual a proportional error, which is also how an operator reads it: “this building is drawing forty per cent more than it should.”

The window ends at the previous observation. Including the point under test in its own reference statistics pulls the median toward it and lets a large anomaly partially explain itself away.

7.4.2 Three floors, each protecting against a different degenerate case

Floor	What it prevents
5% of typical load on the <i>denominator</i>	Dividing by an expectation of 0.0015 kW turned an ordinary 0.85 kW reading at midnight into a robust z of 15 319 — a certain alert, ranked above every real fault in the portfolio, generated entirely by the bottom of the overnight trough on a holiday.
1% on the relative <i>spread</i>	A building the forecaster models almost perfectly gets a near-zero MAD, every score becomes undefined, and the detector goes permanently blind on exactly the buildings it understands best — a failure invisible because it produces no output rather than wrong output.
A finite sentinel for a stuck meter	The natural answer is infinity, but an infinity in a float column sorts ahead of every real detection and floods any “most severe” list with frozen meters.

The flatline sentinel is derived from k rather than fixed, and that matters: it was once a constant 6.0, chosen when k was 3.0, where it cleared the “high” band at 4.2. Tuning later moved k to 8.0 and the band to 11.2, so every stuck meter had since been filed *medium* — the lowest tier, alongside a marginal blip. A frozen meter is a certain hardware fault and a quarter of the injected faults; filing it lowest is backwards, and it meant a critical-only webhook would never forward one.

Severity is graded against k for the same reason. With absolute bands at 5 and 8, a detector running at $k = 8$ reports every detection as critical, because nothing below 8 is flagged at all.

7.4.3 A separate detector for a stuck meter

A flatline is invisible to the residual test by construction: a constant series has zero spread, so its robust scale collapses to the floor and the deviation never resolves. On the seeded portfolio flatlines are 116 of 450 injected faults — a quarter of everything there is to find — so running only the residual detector caps recall at roughly three quarters no matter how k is tuned. The flatline check is trivial and it is a separate function rather than a special case inside the statistics.

7.5 Scoring: the unit is the alert

Both precision and recall are measured in **episodes**, and getting that right mattered more than any change to the detector.

An earlier version counted recall over events but precision over individual flagged hours. Mixing units makes the F1 meaningless: a single real ten-hour fault flagged for six hours scores

as six predictions rather than one correct alert, so precision is divided by a number with nothing to do with how many times anyone was told something. Measured on the seeded portfolio, that mis-specification alone reported precision around 0.19 for a detector whose episode precision is far higher.

Consecutive flagged hours are grouped into one episode; episodes separated by less than three hours are merged, because a fault whose signal dips below threshold for an hour is still one fault. Overlap with ground truth is tested with a three-hour tolerance, because lag features carry a disturbance for hours after the event ends, so a flag shortly after a real fault is a late alert rather than a false one.

Alerts outside the ground-truth window are discarded, not counted.

Data keeps arriving at 720×; the truth file does not. Scoring the whole series against a truth window that stopped months of data time ago counts every correctly detected later fault as a false alarm — measured on this stack, it read episode precision as 0.26 where the detector’s actual precision was 0.50. Coverage is tracked *per run*, not as one span from earliest to latest: the seeded window and the live windows are separated by however long the simulator ran while its ground-truth file was going nowhere, and a single min-to-max span would swallow that gap and quietly resume counting correct detections inside it as false alarms.

7.6 The threshold study

Table 7.1: Threshold sweep behind the choice of k , from `python -m gemp.ml.evaluate`. Measured on an earlier portfolio and reported for the shape of the trade-off rather than as a current figure; the current operating figures are in §6.7.

k	Alerts	Precision	Recall	F1
3	2 986	0.442	0.928	0.599
4	1 522	0.703	0.913	0.794
5	1 127	0.827	0.875	0.850
6	958	0.879	0.823	0.850
8	757	0.945	0.768	0.847
10	648	0.968	0.717	0.824

7.6.1 A negative result that was corrected rather than deleted

For most of the project this section reported that the 0.6 precision gate was unreachable by any means. **Half of that was right and is retained.** It is right that no threshold reaches it: k only slides a point along one curve, and 64 combinations of k , minimum episode duration and minimum peak z did not move the curve either. Requiring longer episodes bought precision at about the same exchange rate as raising k , and requiring a higher peak z made precision *worse* ($0.607 \rightarrow 0.535$ at $k = 8$) — because a frozen meter barely deviates from its expectation at all, while the largest residuals are legitimate load the forecaster failed to anticipate. Residual magnitude does not separate real faults from forecast misses.

The other half was wrong. Treating 0.55 as a property of the problem was the error; it was a property of the *forecaster*, and the ratio-to-profile feature moved it. The two suppression knobs are kept in the code, off by default, so the failed attempt is reproducible rather than folklore.

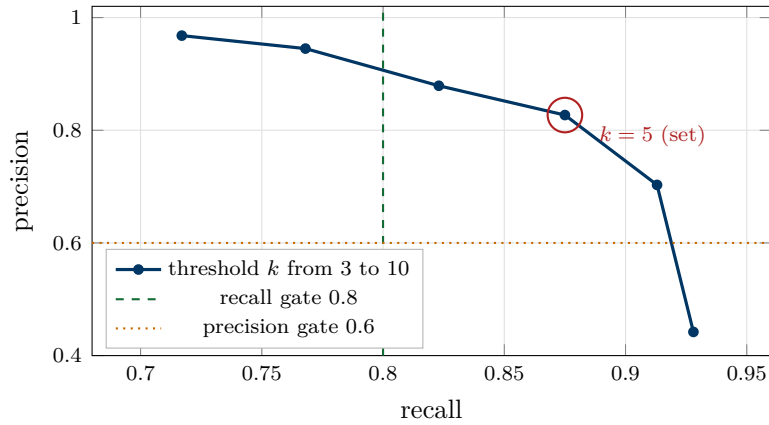


Figure 7.1: The curve traced by k , with the operating point circled and both review gates marked. $k = 5$ and $k = 6$ tie on F1; $k = 5$ is chosen because a missed fault burns energy every hour it runs while a false alert costs one look at a chart, and because it keeps a margin above the recall gate rather than sitting on it. The flag rate at $k = 5$ is 0.88% of all hours.

Two of the numbers the paper could have quoted were measurement artefacts.

The forecaster was predicting down to -1.32 kW in the overnight trough and the detector divides by the expectation, so an ordinary 1 kW reading scored a robust z of 15 319. And a simulator that had silently rewound its data clock by thirteen months filled the ground truth with 1 762 phantom events out of 2 212, reading as recall 0.375 for a detector whose recall is 0.84. Both are fixed, both are pinned by tests, and the rule that came out of it is written into the parameter file: anything written about **F9** must come from a run after this, not from the earlier sweeps.

7.7 Two detectors, one statistic

The nightly job re-scores the whole portfolio, which is the right place to tune thresholds and measure quality. It is the wrong place to notice that a chiller has been running since midnight: a fault found at 03:17 the next morning has already burned a night of energy, and during a demonstration nothing would appear on the map at all.

So a streaming detector scores each reading as it is written, using the same statistic with a seasonal-naive expectation — the reading 168 hours earlier. That choice is practical: the learned model predicts from features that include lags of the value being predicted, so scoring a reading the instant it arrives would mean running the whole feature pipeline per reading, where last week’s same hour is one array lookup needing no model loaded. Memory is bounded by construction — 168 hours of readings plus 30 days of residuals per building, a few hundred kilobytes across fifty buildings — and the dictionary holding per-building state is an LRU capped at 512, because its keys come straight out of an MQTT payload and a publisher sending invented building ids would otherwise grow it for the life of the process.

Scoring mutates memory, so it has to be rolled back when a batch is not stored.

The residual window that judges the next reading, the flat-run counter and the ring of recent values all advance during scoring — and scoring happens while the readings are still uncommitted, because a flatline run has to be counted across the batch in order. A failed commit therefore left the detector having already seen readings the database does not have; the ingester requeues that batch by design, and the second pass appended every residual twice into the window the next reading is measured against. A doubled window is a *wider* window, so real faults stop being flagged — the statistic drifts away from the data in the one direction nobody checks. The fix is a snapshot per building touched, restored on any exception, which is the same rule the chain heads already follow: memory that describes stored rows may only advance when the rows are stored.

Chapter 8

Tamper-evident storage

8.1 The threat, stated narrowly

The threat this layer addresses is specific and is taken from the proposal’s own framing: *consumption figures quietly edited after collection to change a reported outcome*. In an energy-audit context that is the attack that matters, because the whole value of an audit is that its numbers were not adjusted afterwards.

The proposal signed each reading independently with HMAC-SHA256. **F5** is that independent signatures detect **modification** but not **removal**: an adversary with database write access and no key can delete inconvenient readings, and every remaining row still verifies perfectly. Selective deletion achieves the same end as editing and is the easier attack.

8.2 Chaining, at the same cost per row

$$\text{sig}_n = \text{HMAC}_K(\text{sig}_{n-1} \parallel \text{canonical}(\text{row}_n)) \quad (8.1)$$

Any modification, deletion or reordering breaks every signature after it, and carrying an explicit sequence number *inside* the signed payload lets the verifier say *which* of the three occurred rather than only that something did.

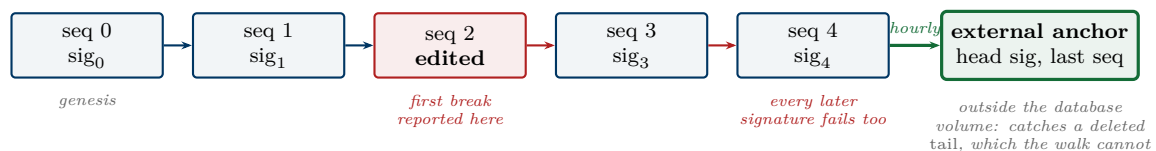


Figure 8.1: The chain, and the one thing it cannot do alone. A modified row breaks the walk at exactly that row and every signature after it, so the verifier reports the *first* break and can name it as modification, deletion or reordering by comparing the sequence it expected with the one it found. Truncating the most recent rows breaks nothing — there is nothing after the deleted tail to check — which is why the chain head is also written hourly to a file on a separate mount. A truncated table then contradicts the anchor.

8.3 Canonicalisation is where silent failure lives

The signed payload covers five fields and the encoding is deliberate in two places that look like formatting choices and are not.

```
payload = {
    "building_id": str(row["building_id"]),
    "ts":         epoch_seconds(row["ts"]),           # integer, not ISO
    "kw":         f"{float(row['kw']):.4f}",          # fixed precision, not repr
```

```

    "source":      str(row["source"]),
    "seq":         int(row["seq"]),
}
return json.dumps(payload, sort_keys=True, separators=(",", ":")).encode("utf-8")

```

Signing the `repr` of a float would make verification depend on platform float formatting, and signing an ISO string would make it depend on how the driver chose to render the offset. Both fail *silently and intermittently* — the worst possible failure mode for an integrity layer, because it produces a tampering alarm on data nobody touched. An integer instant and a fixed-precision decimal each have exactly one representation.

Only five fields are covered, and that is a decision rather than an oversight.

Anything added to the table later is explicitly *not* protected unless it is added here too — which is why the field list is a literal rather than a loop over the row's keys. A loop would silently extend the signature's scope on the next migration, and a signature whose coverage changes without anyone deciding it should is not a guarantee.

8.4 The sequence is the ingester's to assign

The MQTT payload deliberately does *not* carry a sequence number. Sequence is assigned by the ingester, which owns the chain. A device cannot be trusted to number its own readings monotonically across a reboot, and letting it try would put chain integrity at the mercy of the least reliable component in the system.

Three consequences follow and all three are deliberate. A device cannot corrupt the chain by restarting its counter. A duplicate delivery must be dropped *before* a sequence is allocated — MQTT QoS 1 is at-least-once, so redelivery is normal rather than exceptional, and allocating a sequence to a row that then loses an ON CONFLICT race would leave a gap the verifier correctly reports as a deletion. And sequence order is arrival order, which coincides with time order because the stream is monotonic per building.

A discarded row once punched a permanent hole in the chain.

The insert is ON CONFLICT DO NOTHING, so a row whose (building, timestamp) already exists is discarded by the database without complaint — while the in-memory chain had already consumed a sequence for it and signed the *next* reading against its signature. One discarded row left seq 10, 12, 13, with 12 chained to a signature that was never stored, which the verifier correctly reports as tampering. That is the worst possible false alarm for the one feature whose entire purpose is to be believed. The fix was to sign *inside* the write transaction after asking what is already stored, and to advance chain state only after the commit returns.

8.5 Verification: three checks, and the order matters

`verify_building_chain` returns three independent verdicts:

Check	What only it can detect
<code>chain_ok</code> — walk the signatures	modification or reordering inside the chain
<code>checkpoint_ok</code> — compare with the external anchor	truncation of the tail, which the walk cannot see
<code>anchor_matches_database</code>	a database whose own checkpoint table has been edited to match a forged chain

The database’s own copy of the checkpoint is deliberately *not* trusted on its own: it lives in the same volume as the data it vouches for, so anyone who can truncate **reading** can truncate it too. It is read in order to be *compared* against the external file, because a disagreement between the two is itself evidence and neither copy can produce that signal alone.

The anchor is read before the rows, and the order is load-bearing.

Ingestion continues while verification runs. Reading rows first leaves a window in which the ingester flushes new readings and then checkpoints past the head the snapshot contains — so the anchor claims a sequence the rows legitimately do not reach, and an intact chain is reported as truncated. That is not hypothetical: the claims harness reported exactly one such false positive against the live stack the first time it ran, on a building nobody had touched. Reading the anchor first makes the check one-sided in the safe direction, because rows written afterwards only ever extend the chain and a chain longer than its anchor is what healthy looks like. A false alarm about tampering is worse than no alarm: it burns a demonstration hunting an attack that never happened, and it teaches whoever is watching to discount the next one.

8.5.1 The verdict names the likely cause

A break at the very first row is reported differently from a break anywhere else, because the most likely explanation is different:

the first row itself does not verify, so the signing key is probably not the one these rows were written with — check `GEMP_HMAC_KEY` before concluding the data was modified

An attacker who can edit the database has no reason to start at row zero. This exact situation occurred during Phase 4: an integration test picked up the SQLite suite’s test key from the process environment and reported a clean production chain as “modified at seq 0”. Saying “modified” there sends whoever is debugging to look for tampering that is not in the data.

8.6 What this does not claim

Two limits are stated in the module rather than left implicit, and they are the reason the proposal’s own narrow scoping of this feature was retained as written.

This is not tamper-proofing against an adversary with host access. Such an adversary can read the injected key and re-sign whatever they like. The guarantee is *tamper-evidence* against an adversary with database write access and no key.

It does not authenticate a reading at its origin. A compromised sensor publishing false values produces records that are correctly signed and factually wrong. Closing that needs per-device signing keys in hardware; closing the first needs replication to a separate machine. Both are outside this build and are named as such.

Integrity is not accuracy, and the platform says so on the screen.

A verified chain says nobody altered what the meter sent. Whether the meter itself behaved is a separate question — and it is the question the anomaly layer answers. The two are presented as different guarantees in the interface rather than blended into one reassuring claim, because a reader who conflates them would over-trust the result.

8.7 Where the key lives, and what that costs

The signing key is an environment secret, never written to the database volume and never committed. `deploy.sh` refuses to run without it and warns if the file is not mode 600. The consequence is stated plainly in the operations documentation: nothing provisions or rotates it, and regenerating it invalidates every existing signature. That is a deliberate property rather than a bug — a key that could be rotated without invalidating history would not be binding history — and it means the key must be backed up *with* the data it signs. The backup captures three things for exactly this reason: the database, the anchor file, and the environment file holding the key. Restoring any two of them is not a restore.

Chapter 9

The web application

9.1 The constraint that produced the architecture

F13 requires the demonstration to survive an unplugged network cable. That single requirement determines the whole front end: no framework, no build step, no bundler, no content-delivery network, no web font, no icon library, no charting library and no map tile server. The files nginx serves are the files in the repository, so what is reviewed is what runs.

The requirement is enforced by the browser rather than by discipline. The server sends a Content-Security-Policy of `default-src 'self'` with no `'unsafe-inline'`, so an external `<script>` added later “just for one icon” is *blocked*, not merely unwise.

Three things the policy forbids that are easy to walk into.

Inline scripts and inline event handlers are discarded, so `onclick="..."` in markup silently never fires. So are `style="..."` *attributes* — `style-src 'self'` governs style attributes as well as `<style>` elements, and three of them had been sitting in the map module since the dashboards were built, doing nothing: the narrative modal’s cards never had the side-by-side layout their markup asked for. Nothing reports this; the attribute is simply thrown away. And a `<style>` element injected by script is blocked too, which once made a measurement probe report that a rule was not applying when the policy was doing its job. Every visual decision therefore has to be reachable from the stylesheet through a class.

9.2 The shell is deliberately empty

```
<div id="root">
  <div class="login-wrap"><div class="loading">Loading&hellip;</div></div>
</div>
<script type="module" src="js/app.js"></script>
```

Every screen is rendered by the application *after* it has asked the server who the caller is. An unauthenticated browser is therefore never handed markup describing data it may not see — no half-drawn dashboard behind a login overlay, and nothing in the page source worth reading before signing in. That is a security property achieved by a structural choice rather than by hiding elements with CSS.

9.3 The nine screens

The map is the most demonstrated screen and the one with the most engineering in it. Its projection, rendering, pan/zoom and footprint switching are working code to be respected; its surroundings are not sacred.

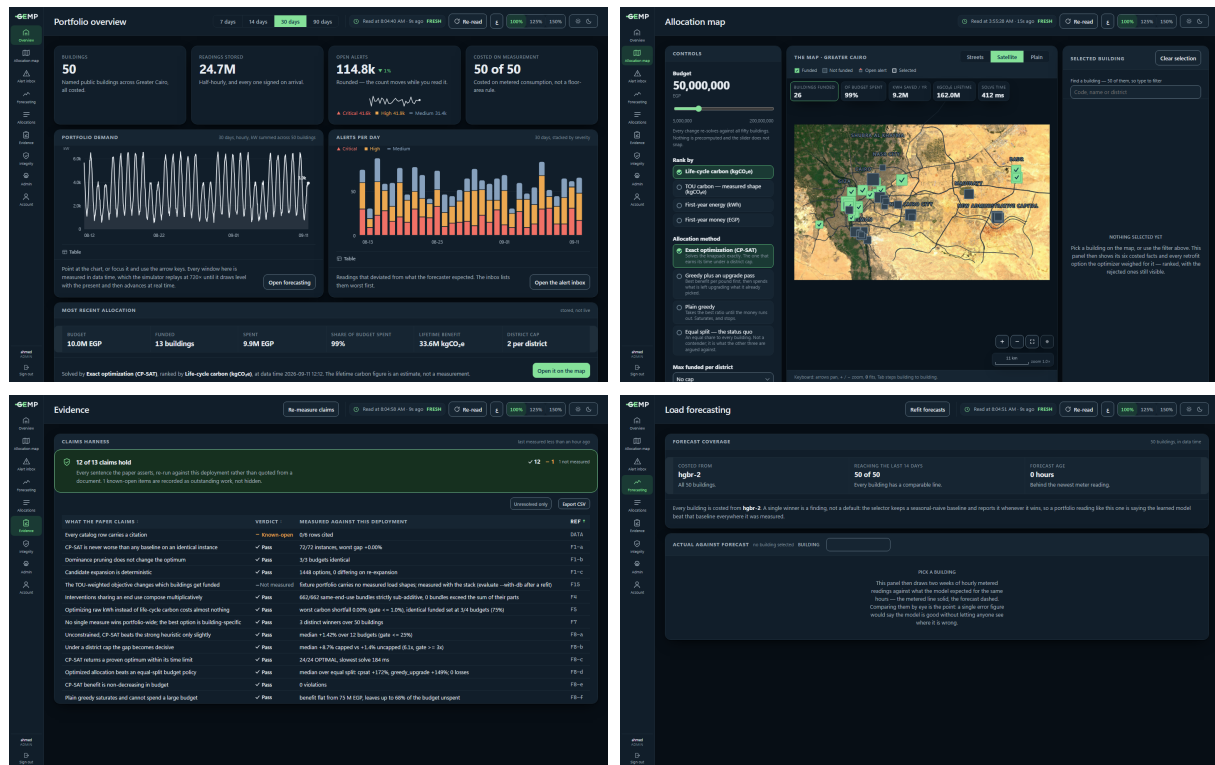


Figure 9.1: Clockwise from top left: the portfolio overview, the allocation map, the evidence screen carrying the claims harness output, and load forecasting. Captured from the development harness against representative data, so the figures in them are illustrative — the claims the platform makes about itself are measured live on the evidence screen. Every window is measured in *data* time, which the simulator replays at 720× until it draws level with the present.

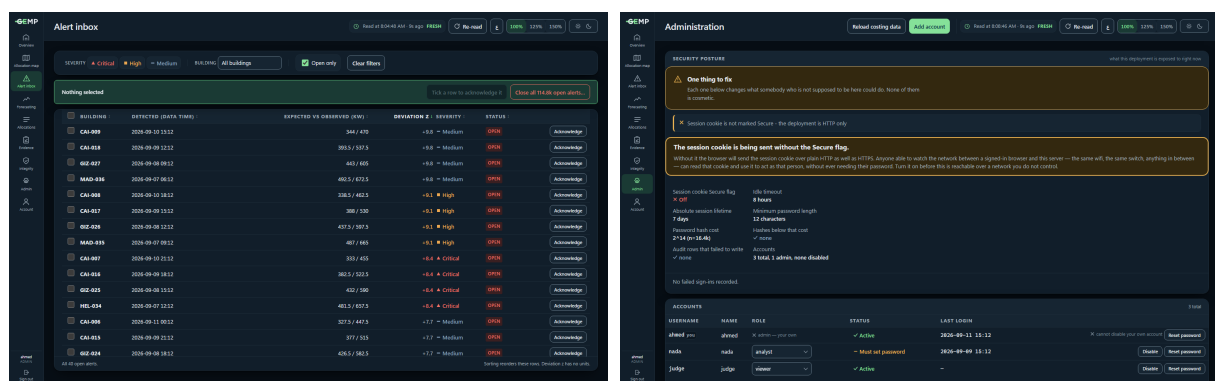


Figure 9.2: The alert inbox, worst first, with bulk and reversible acknowledgement; and the administration screen, which reports the security posture of the running deployment alongside accounts and the audit log. Acknowledgement being reversible is what stops the open count from being a number that only ever grows — sixteen months of replay had accumulated over eleven thousand critical alerts that nobody could clear, because the column existed and nothing could set it.

9.4 The map, and why it has no tile server

Fifty polygons is inline SVG. A mapping library’s default tile layer fetches from a remote host, which would be blocked by the policy and blank on a demonstration machine with the cable out. So the satellite ground is *one JPEG committed to the repository*, fetched by a script and refreshed by hand, and the street context is a GeoJSON file baked in at build time by a second script (about 970 kB: 1 695 roads, 890 neighbouring buildings, 14 water and park features).

Four details in that layer are load-bearing:

- **One projection.** The context is projected with the portfolio’s own `toScreen`. Two layers on two projections would drift apart at zoom.
- **Panning moves a transform; it does not re-render.** The context is roughly 2 600 paths and rebuilding that markup on every pointer move made the map unusable to drag. Measured: 6 ms to pan, about 20 ms per zoom step, because zoom still goes through a render — the zoom band decides what is drawn.
- **Detail is gated by zoom band.** At the opening view the minor streets are a grey wash hiding the roads anyone would navigate by, and neighbouring footprints are specks; neither is drawn until it means something. Verified: far = 0 minor roads and 0 context buildings, mid and close = 1 384 and 890.
- **The imagery is drawn outside the zoom transform**, so zooming samples the file at the size it is shown at rather than magnifying a raster the browser already made. That created its own bug — the streets slid off the photograph on every drag, because the fast pan path moved the group and not the image — and both paths now go through one piece of arithmetic, pinned by a test.

The map claims every touch that lands on it, and is capped so that is safe.

A pannable map and a scrollable page want the same one-finger drag, and only one of them can have it. `touch-action: none` on the SVG gives it to the map — without it the browser takes the gesture for a page scroll and sends `pointercancel` a hundred milliseconds in, which is a map that can be tapped and never dragged. The price is that a finger landing on the map can no longer scroll the page, so on a phone the stage is capped to a fraction of the visible viewport rather than filling it: there is always page above and below to swipe from. An element that swallows every vertical swipe *and* fills the screen is a scroll trap.

9.5 Charts drawn by hand, and what a reader can do with them

The charts are inline SVG computed in the browser, because a charting library from a CDN would be blocked rather than merely unwise. Each chart is a host with three parts, and only the first is ever redrawn:

<code>.chart-plot</code>	the SVG — replaced on every resize
<code>.chart-tip</code>	the readout — positioned from a custom property, never rebuilt
<code>details.chart-table</code>	the same numbers, collapsed

Listeners bind to the *host* once and read metadata the latest draw left behind. Binding to the SVG would mean re-attaching after every resize, and a missed re-attach is a chart that silently stops responding at one particular window width.

A reader can point at any chart and get a crosshair snapped to the nearest reading with one readout listing every series at that moment — so the pointer never has to land on a line. They can focus it and use the arrow keys for the same values, with Shift stepping ten at a time. And

they can open a `<details>` table of the same numbers, thinned past sixty rows and labelled as thinned, which needs no script and is announced for free.

A colourblindness check changed the charts, and the real fix was refused.

Run through a deuteranopia check in the light appearance, `--sev-critical` and `--sev-high` measure ΔE 0.6 apart — to roughly one man in twenty they are the *same colour*, and the alerts chart stacks them on each other. `--sev-high` against `--sev-medium` is 13.0 in normal vision, below the 15 at which two marks can be told apart at a glance by anyone.

The real repair is re-stepping the three tokens, and that is *not a chart's decision to make*: those colours are also the alert badges, the map markers and the severity chips, so they change everywhere or nowhere. Until somebody does that, the charts stop relying on hue alone — each band carries a hatch at 45°, 135° and 90°, and the legend keys carry the same textures, or the legend would be the one place a reader still had to separate bands by hue. The workaround is labelled as a workaround; the token re-stepping is still open.

Two related defects were fixed in the same pass. The proportion bar was painting a *categorical* question — which forecaster is in use — in the severity ramp, so a portfolio entirely on the current model drew a full-width bar in critical red with “100%” on it: an alarm about good news. And the readout puts the value first and heavier with the label after it, which is the legend's hierarchy inverted, because at that point the reader already has the series and wants the number.

9.6 Two appearances, and tokens written three times

Colour tokens are semantic — named for the job, not the colour — and each is written three times: once for light on bare `:root`, once under `@media (prefers-color-scheme: dark)` guarded by `:not([data-theme="light"])`, and once under `:root[data-theme="dark"]`.

That is not redundancy to tidy up.

A token defined *only* inside a media query does not exist for anyone outside that query, and without the third block an explicit dark choice on a light system does nothing. The appearance switch offers Light and Dark; “Auto” has no button but is the default *state*, and it means no `data-theme` attribute at all, leaving `prefers-color-scheme` in charge. That ordering is forced by the policy: nothing can run before first paint, so a design whose correct appearance depended on JavaScript would flash the wrong one on every load.

Every text pair was measured rather than eyeballed. On the panel surface the primary ink measures 18.4:1 in light and 14.8:1 in dark; the secondary 7.0 and 8.6; the tertiary 5.3 and 5.6. A separate `--control-line` token exists because a divider may be faint but the boundary of an input may not: it measures 3.5:1 on white, which is what WCAG 1.4.11 asks of a control edge.

Where colour carried meaning alone, something else now carries it too: the second chart series is dashed as well as amber, a funded building has a lighter outline as well as a green fill, and each severity pill has a dot.

A documented contrast ratio is against one surface, not all of them.

The tertiary ink is annotated 4.53:1 in the stylesheet, and that holds against a white panel. On the page ground, one shade darker, it measures 4.12:1 and fails. Measure the pair actually being shipped.

9.7 Two languages, and the defect that shipped

The interface renders in English and Arabic, with direction and language stamped on the shell and the layout built from CSS logical properties rather than physical ones — thirty-five declarations were converted for it.

The interesting part is how the first attempt failed. A translation helper that returns the *key* when a string is absent is a reasonable fallback until the catalogue ships with an Arabic table and no English one: then the *default* language rendered `nav.overview` in the navigation rail and `shell.signout` on the button. Three more keys were in neither table. Nothing else noticed — the ids existed, the paths resolved, the suite was green — because no other test read what a control *says*.

The regression test that came out of it is one of the more useful in the repository: it extracts every key the interface calls, including the prefixes of keys built at run time, and asserts that every language has all of them *and* that no translated key is unused.

9.8 The interface’s contract, enforced by tests

`tests/test_web.py` is 33 test functions that render nothing — there is no headless driver — but pin the two things that actually break a demonstration: an endpoint changing shape underneath the interface, and a stray external reference. What it enforces:

Pinned	Because
No off-origin reference in any served file	F13. A verification that ran once protects nothing
No bundler or package manifest	the files served are the files reviewed
Every id a script reaches for exists	a typo in an id is silent: the handler never fires and the control looks fine
Every API path the interface calls is a route the server serves	keeps the two from drifting apart silently
<code>.modal[hidden]</code> sets <code>display:none</code>	an author <code>display</code> rule beats the browser’s <code>[hidden]</code> ; both modals once rendered on load and the map opened behind an overlay reading “Solving...”
The zoom ceiling is at least 4× the footprint threshold	real footprints switch in where the median outline is 6.4 screen pixels; correct geometry nobody can read is a claim rather than a demonstration
The interface never says “three methods”	there are four solvers; stale copy reads as a system nobody checked
The map never solves for a role that may not	a viewer’s first sight of the product was a red error
Window listeners are tied to a lifetime	five visits to the map meant five resize handlers
Credentials are never minted through a browser prompt	clear text, no confirmation field, no password manager
Every hover rule is gated behind a hovering pointer	a touch browser leaves the last tapped thing in <code>:hover</code>
The page never forbids pinch-zoom	WCAG 1.4.4, and the first thing a low-vision reader reaches for
Every table can scroll inside its own box	six columns of figures is about 560px; in a dialog that width becomes the dialog’s
The map quotes the grid factor the optimiser actually uses	the panel prints it from a constant, and it drifted once when the factor was cited

The last one is the pattern worth copying.

The evidence panel prints “benefit is kWh saved $\times$ lifetime $\times$ factor” from a hardcoded string rather than from an API call, because making one line of explanatory prose depend on a second request that can fail is a bad trade. That is defensible and it has exactly one cost: the string can drift from the parameter file. It did, silently, when the factor was cited. The test reads both and fails if they disagree — which is the cheap half of keeping a system honest that asks its readers to check it.

9.9 The phone, which found nine defects in one afternoon

The product was carried to a phone for the first time on 2 September and could not be used on one. The map answered a finger with a synthetic click and nothing else; the navigation rail took a fifth of a 375px screen; every control had been sized against a cursor. None of that is visible from a desktop, which is exactly why it survived to a week before submission.

The repairs: Pointer Events throughout, so mouse, pen and touch go through one implementation of a pan rather than a mouse one and a touch one drifting apart; `pointercancel` handled beside `pointerup`, because an incoming call or the notification shade ends a touch that way and never sends `pointerup`, leaving the map mid-drag so the next tap teleports it across the city; one named clamp shared by the buttons, the keys, the wheel and the pinch; the rail becoming a fixed bottom bar with a matching pad and safe-area inset on the content, so it stops covering the last row of every table.

Three touch targets remain under the 44px AAA floor and are documented rather than quietly left: row-selection checkboxes at 28px, a column sort control at 42px and the show-password button at 40px. All three clear the 24px WCAG 2.5.8 minimum, and each was left short deliberately — a 44px checkbox is as tall as the row it selects, a 44px sort control makes a sticky table head 56px deep on an 812px screen, and a 44px reveal button inside a 44px password field has nowhere to sit.

9.10 Accessibility work that is not decoration

A skip link that is focusable at all times — `display: none` would make it unreachable, which is the usual way this control gets broken — targeting an element with `tabindex="-1"` so the fragment moves focus rather than only scrolling. One `:focus-visible` ring across the whole application, where previously only the login inputs had one, which had made every dashboard unusable from a keyboard. The active navigation item carries `aria-current="page"`, and that attribute is also what styles it, so the visual state and the announced state cannot drift. Selection checkboxes name the row they select, because a hundred rows each announcing “Select this row” tells a screen-reader user that there is a checkbox and nothing about what ticking it would do. Dialogs trap focus and return it to whatever opened them, with Escape handled on the modal rather than on the document so it adds no listener to leak.

Chapter 10

Security architecture

10.1 The shape of the thing

Two layers, and the split is the point.

Authentication is enforced by middleware, not by remembering to add a dependency. Everything under `/api/` requires a session unless its path is on an explicit public list. A route added next month is protected the moment it exists, by somebody who has never read the auth module — which is the only kind of default that survives a deadline.

Authorisation is per route, because only the route knows whether it reads or writes. `Depends(require(Role.ANALYST))` is the whole vocabulary, and the dependency carries the requirement as an attribute so a test can read what each route asked for rather than trusting that somebody remembered to ask.

The public list is deliberately short and every entry is a decision:

Path	Why it is public
<code>/health</code>	a readiness probe cannot hold a credential; it reports dependency status only, never data
<code>/api/v1/auth/login</code>	logging in without being logged in is the point
<code>/api/v1/auth/logout</code>	—
<code>/api/v1/auth/session</code>	the application asks who the caller is on load; a 401 for the ordinary “not signed in yet” case would put a red line in every console
<code>/api/v1/pass/check</code>	the Judge Pass page, reached from a QR code by somebody with no account. Both demand the shared code before answering anything
<code>/api/v1/pass/redeem</code>	—

10.2 Sessions, not tokens

Sessions rather than JWTs, and the reasoning is short: a JWT cannot be revoked, and `localStorage` hands it to any cross-site scripting bug. An opaque token lives in an `HttpOnly; SameSite=Lax` cookie and **only its SHA-256 is stored**, so a database dump, a stray backup or an injection that reads the session table yields values that cannot be replayed.

Property	Value	Why
Token	32 bytes CSPRNG	—
Stored form	SHA-256	not script: the input is 32 bytes of CSPRNG output, not a guessable secret, and
Idle timeout	8 hours	a session left open on a shared machine stops working
Absolute lifetime	7 days	a stolen cookie that <i>is</i> being used cannot renew itself indefinitely, which an idle
Touch interval	1 minute	writing <code>last_seen_at</code> on every request turns a read-only page into a write on

The token is regenerated on login, never adopted from what the client supplied, so session fixation is impossible: an attacker who plants a cookie value before login finds it replaced the moment the login succeeds. A password change revokes *every* session for the account — a password change is the action somebody takes when they believe a credential is compromised, and leaving the attacker’s session alive would defeat the point.

10.3 Passwords

scrypt from the standard library, and the dependency argument is the whole reason: a security-critical primitive that arrives with no new package cannot fall out of date because nobody ran an upgrade, cannot ship in a stale wheel, and cannot disagree with itself across a container and a laptop. It is memory-hard, which is the property that matters against GPU cracking.

Parameters are stored *per row* rather than as a constant, in a self-describing string:

```
scrypt$n=16384,r=8,p=1$<salt-hex>$<hash-hex>
```

Hardware gets faster, and a deployment that hardcodes its cost factor can never raise it without invalidating every password it holds. Storing them means an old hash stays verifiable while new ones are written at a higher cost, and the upgrade happens on a successful login — the only moment the plaintext exists.

Policy follows NIST SP 800-63B: a 12-character minimum, no character-class requirements, a maximum long enough for a passphrase, no periodic expiry, and a small deny list that only catches the handful that would otherwise appear in a demonstration account. Verification is constant-time, and a malformed stored value returns false rather than raising — a corrupted row must fail the login, not return a 500 that tells an attacker the account exists and something is wrong with it specifically.

The administration screen reports how many hashes are below the current cost.

A hash is upgraded only on its owner’s next sign-in, so an account that has not logged in since the cost was raised keeps its old, cheaper hash indefinitely. Two things follow, and the second is the one worth surfacing: the password is protected at the weaker factor, and verifying it is measurably *faster* than the deliberately equal-cost work done for an account that does not exist — which is the timing side of the account-enumeration defence. Neither is visible anywhere, so neither gets acted on. A security control nobody can see the state of is a control nobody maintains.

10.4 Login as an enumeration and denial-of-service surface

Authentication returns an identical failure for “no such user”, “wrong password”, “account disabled” and “account expired”. Distinguishing them turns the login form into an account-enumeration oracle, and knowing which usernames are real is most of the work in a credential-stuffing campaign. The timing side is closed too: the code hashes even when the user does not exist, against a well-formed hash of a value nobody knows, because returning early would make a missing account measurably faster to reject.

The throttling design is the most interesting security decision in the project, because the obvious version is an outage lever.

A per-IP lockout behind a reverse proxy is an unauthenticated denial-of-service.

Every request reaches the application from nginx's own address, and a public ingress collapses every visitor to one address. Keyed on that, **five failed logins lock the login form for everybody** — proven live during a pre-exposure penetration test: five sprayed failures and the real administrator got a 429. Worse, a throttled attempt raises before it records, so the window decays from the last *recorded* failure: fifty requests every fifteen minutes sustained a total login outage indefinitely.

A later revision re-armed a *shared* bucket at fifty failures, which reintroduced the same outage at ten times the price. The final design makes the shared bucket cost **time rather than access**: once armed, each attempt from the shared address pays a one-second delay before its password is checked, which drops spraying from roughly six hundred attempts a minute to under sixty — the same order of reduction the lockout was reaching for — and nobody is ever refused. The delay is capped by a semaphore at eight concurrent sleepers, because a delay holds a threadpool worker and an unbounded number of them is its own outage; past the cap the slowdown is skipped rather than queued. Per-username lockout is untouched and remains the control that actually stops guessing a known account: five attempts per username per fifteen minutes, from any address.

The counter store is bounded, which it was not at first: keys are attacker-chosen (`user:<whatever was typed>`) and nothing removed them, so a login flood with a fresh username each time grew the dictionary for the life of the process. A sweep now drops keys whose window has aged out, deferred until there are enough keys for the $O(n)$ pass to be worth doing.

The password-change endpoint sits under the same counters, because it is a password guess against a known account: somebody who has got hold of a session on a shared machine could otherwise grind the current password at whatever rate script allows, and that secret is the only thing between them and permanent ownership of the account.

10.5 CSRF, and one deliberate exemption

A double-submit token: a value in a readable cookie that must be echoed in a header. An attacker on another origin can cause the cookie to be *sent* but cannot *read* it, so they cannot produce the header. `SameSite=Lax` already stops the classic cross-site form post and every mutating endpoint consumes JSON, so this is the third layer. The comparison is constant-time.

The auth endpoints are exempt, and the exemption is load-bearing rather than convenient.

A browser holding a stale session cookie and no CSRF cookie — an expired session, a revoked one, a cookie left over from a previous deployment — would otherwise have its login POST refused, leaving the user unable to authenticate and unable to see why. They would be locked out by the security control rather than by the security decision. What that gives up is protection against login-CSRF, where an attacker forces a victim to log in as *them*; `SameSite=Lax` is what covers it, because a cross-site POST carries no cookies at all.

10.6 Roles, and the audit trail

Three ordered roles — `viewer < analyst < admin` — ordered rather than a permission matrix because the distinctions are genuinely hierarchical and a matrix invites combinations nobody has thought about. An unknown role value is never sufficient: a typo in a migration should lock somebody out, not let them past.

Every privileged action writes an audit row **including the ones that failed**, which is the half most audit logs leave out. The trail is best-effort — a failed audit write must not fail the action — and because a swallowed failure is invisible, the count of failures is surfaced on the administration screen. Refusing the action when the log is unavailable would be the stricter choice and for a system moving money it would be right; here it would mean a database hiccup takes the demonstration down, so the trade is named rather than hidden.

Two guards exist so an administrator cannot lock the deployment out of itself: an admin cannot remove their own admin role, and cannot disable their own account. Disabling somebody else revokes their sessions immediately, or the account keeps working until its cookie expires up to a week later.

10.7 Everything else the middleware does

Security headers are sent on every response, and each closes a class of attack a single-origin application is otherwise still exposed to: a strict CSP, `X-Frame-Options: DENY, nosniff`, `Referrer-Policy: same-origin` (map URLs carry run ids), `Cross-Origin-Opener-Policy`, a `Permissions-Policy` denying geolocation, microphone and camera, and `Cache-Control: no-store` — because caching a page rendered for one user and serving it to the next is a data leak that looks like a performance optimisation. HSTS is added only when the Secure-cookie flag is on, since sending it from a plaintext origin is ignored by browsers and would strand a demonstration on http.

There is **no CORS configuration anywhere**, because the API is served from the same origin as the page. Saying so means a misconfiguration cannot turn into one.

Unhandled errors return the exception *type* and nothing else: a stack trace is a gift to anyone probing the service, and an empty body is useless to whoever is trying to fix it during a demonstration.

Two resource limits protect the machine and the moment. Solving is CPU-bound and CP-SAT takes eight search workers, so concurrent solves are capped globally *and* per-account — without the second, one analyst scripting the endpoint keeps every slot busy with requests that are each legally under the global cap, and the demonstration's own slider request is the one that starves. Excess requests get 429 rather than a queue, because during a demonstration the request that matters is the one happening now.

10.8 The independent security review of 12 September

A full offensive and defensive review was conducted on the revision this report documents. Method: static analysis (Bandit over `src` and `scripts`, 0 findings at any severity; the 20 in-source suppressions each read and confirmed legitimate), dependency audit, a line-by-line manual read of every service, a secret scan across all 132 revisions of history, and live black-box probing of the deployed host.

Verdict: no exploitable defect was found in the application code or the infrastructure. Table 10.1 lists everything that was found; all of it is governance or hardening.

10.8.1 What the live host verified

- **Reachability.** All eighteen probed TCP ports on the public address are closed or filtered. The cloud firewall carries *zero* inbound rules and the host firewall admits only the private-network interface — two independent controls, which matters because a container's published port bypasses the host firewall and the cloud rule is the only thing refusing it.
- **Provenance.** The running image is the signed registry digest built by the pipeline, not a local build.

Table 10.1: Security review findings. Severity reflects real-world reachability at the time of review, not textbook category.

ID	Sev.	Finding	Disposition
S-1	Medium	<code>main</code> has no required status checks and no required review, and auto-deploy was on — so an unreviewed push that passed CI reached the public host	Auto-deploy frozen after the final change; branch protection recommended
S-2	Low	Dependabot alerts, secret scanning and code scanning disabled at the repository level, so the pipeline’s SARIF upload no-ops	Recommended; in-pipeline scanners still block
S-3	Low	The VM permits SSH password authentication and <code>sshd</code> listens on all interfaces — unreachable from the internet behind the NSG and firewall	Key-only authentication recommended
S-4	Low	No automatic operating-system patching on the host	Recommended for anything longer-lived
S-5	info	The <code>i18n</code> helper does not escape interpolated variables; the <code>DB-init</code> script interpolates a password into a SQL literal; <code>*.pem</code> was not ignored	<code>*.pem</code> fixed in <code>43ef04f</code> ; the other two safe as used

- **Runtime surface.** The container runs as an unprivileged numeric uid and ships no package manager at all — `pip`, `setuptools` and `wheel` are uninstalled in the final image layer, which is what cleared the last three fixable vulnerability findings and also means the advisories a dependency audit reports against `setuptools` are not present in what is deployed.
- **Configuration.** Secure cookies on; interactive API documentation off (both documentation paths return 404); the pass window and daily ceiling set as intended.
- **Headers.** The origin emits the complete set including HSTS; the public ingress strips four of them on the way out, and the one that matters — `frame-ancestors 'none'` in the CSP — survives, so clickjacking remains blocked. This is a property of the ingress rather than a defect in the application, and is recorded as such.
- **History.** No secret has ever been committed: no key material, no credential pattern, and no `.env`, `.pem`, `.key` or password file in any of 132 revisions.

The one credential incident, and what it says about process.

An administrator password was pasted into a chat transcript on 28 August, while the host was reachable over a public ingress. It was treated as compromised: rotated on 11 September through the account screen — which re-checks the current password, enforces the length policy and revokes every session — and the account it belonged to was later deleted outright. The repository was never the leak path; the transcript was. That is the usual shape of a real credential incident, and it is why the review’s scan covered history as well as the working tree.

Chapter 11

The Judge Pass

11.1 The problem this solves

Final judging happens at a booth. A judge arrives, is handed something, and looks at the product on *their own phone*. Everything else in this report assumes an account exists; at a booth none does, and the two obvious answers are both bad. A shared demonstration account handed round a room is a credential with no expiry and no attribution. A sign-up form is a registration surface open to the internet.

The Judge Pass is the third answer: a card carrying a QR code, which opens a page asking for an email address and nothing else. Confirming it creates a viewer account named after that address, signs the browser in *on the spot*, and emails a generated password so the judge can come back on any device until the pass closes.

11.2 One shared code, and where the limits live

Every card carries the same code. That is a deliberate trade, and the alternative was considered first.

A per-card code is a key; a shared code is only a door.

A per-card code means whoever holds that card gets exactly one account, which is tidier — and it means printing, tracking and reconciling fifty distinct codes, and a judge handed a card whose code was already redeemed gets a dead link in front of an audience. Cards also leave the booth in pockets and on photographs, so the number printed was never going to be the real limit anyway. The decision was therefore to make the *system* carry the limits rather than the stationery, and to say so:

Limit	What it stops
A closing time, with a UTC offset	after it the page issues nothing and every pass account stops working, so the card a judge takes home is a souvenir rather than an open door
One pass per address	a second attempt signs nobody in and sends nothing
A ceiling per rolling 24 hours	a photographed card cannot turn the sender into a spam cannon and get the sending account suspended mid-event
An off switch in the interface	an administrator can pause issuing from a phone; pausing signs nobody out, because a judge halfway through the product is not what it exists to stop
A minimum code length	a short code is a guessable one, and a guessable one is an open registration form

Per-address rate limiting is deliberately *absent*, for the reason given in §10: every request reaches the application from one private address, so a per-IP limit would treat the whole venue as one person.

11.3 The flow, and two details that matter

1. The QR code encodes `<public-url>/#/pass/<code>`.
2. The page checks the code before it renders anything. Without the right code the whole answer is “invalid”: not whether passes exist, not when they close, not who to ask.
3. The judge types an email address. There is no password field, no checkbox and no second step.
4. Confirming issues the pass, signs that browser in immediately, and returns.
5. The email is sent *after* the response, as a background task.

The code travels in the URL fragment, and the email is sent after the response.

A fragment is never sent to a server, so the code stays out of every access log between the phone and the application, and out of any `Referer` header. The same reasoning puts the code in a request *body* rather than a query string on both public endpoints.

Sending the email after the response means a slow provider — or none at all — never stands between a judge at a booth and the product. The email is for coming *back*, on this device or another; the pass itself does not depend on it. Because nobody is waiting on it, its outcome has nowhere to be reported, so it is recorded on the pass row where the administration screen shows it.

11.4 What the account is

An ordinary `viewer` with an ordinary script password and ordinary sessions. Its one new property is an end date, and that is enforced in three places rather than one: at sign-in, on every request while resolving the session, and as a cap on the session’s own expiry so a cookie cannot outlive the pass by the difference between one day and seven.

An expired pass fails exactly like a wrong password. A pass that has closed is not a secret, but “this account exists and has expired” is still an answer about which usernames are real.

11.4.1 The generated password

Three words and two digits from a curated list, drawn from `secrets` rather than `random` — for example `cedar-nile-solar-47`. The words are short, common and unambiguous so the password can be read off a phone screen and typed on another device without a copy-paste.

Twenty-six bits, and why that is the right amount here.

Three words from a hundred plus two digits is about 26 bits, which would be indefensible for a durable credential. What makes it proportionate is everything around it: the account is view-only, it dies within the day, and it sits behind a five-attempt per-username lockout. Spending more entropy would buy nothing against that lockout and would cost a judge a mistyped password in front of an audience. The password is generated, hashed into the account, handed to the mail sender in memory, and never stored or logged — the pass row records the *outcome* of the email, never its contents.

The account does *not* carry a forced password change. An admin-issued credential normally does, so a temporary password cannot quietly become permanent — but here the credential is system-generated and dead by the end of the day, and a forced change would put a second form between a judge and the product while protecting nothing.

11.5 The email

One message, bilingual, in whichever language the judge used on the pass page. It is built as tables with inline styles, because that is what every mail client renders, and it carries **no external images** — many clients block them and the mark is drawn in HTML. It doubles as the thank-you: it names the team, states what the pass opens, gives three figures from the portfolio with the poster’s own caveat beneath them, and says when the pass closes in Cairo time.

Every interpolated value is escaped. The address the judge typed is the one attacker- controllable string in the feature, and the validating regular expression excludes whitespace — so SMTP header injection is not possible even before the mail library’s own refusal to accept newlines in a header.

11.5.1 Three transports, and why there are three

The first configured provider wins, in the order Brevo, Resend, plain SMTP. Unconfigured is a normal state, not an error: with nothing set every send reports **off** and touches nothing, which is what the test suite and an offline laptop both want — and the judge is still signed in either way.

The project tried to do this properly and ended up with a Gmail mailbox, for a reason worth recording.

Both API providers require a sender address on a domain the account has verified. The domain the team wanted, `gemp.com`, belongs to somebody else — it resolves to third-party name servers and carries its own mail exchanger — and a free-tier account sending from a consumer address draws a freemail warning and a verification requirement the event’s timescale did not allow. So the transport is a Gmail mailbox with an application password over STARTTLS.

That choice has a real cost and it is stated rather than hidden: mail from a brand-new consumer mailbox lands in Spam until the account has some sending reputation. Two causes that were *ours* were removed — a hidden preview-text block, and a message identifier minted locally claiming the provider’s own domain from a machine that is not theirs, both of which a filter weighs against an unknown sender — and the interface now tells the judge where to look. The rest is the account’s reputation to earn, which is an operational task rather than an engineering one.

The SMTP path never falls back to plaintext: port 465 is TLS from the first byte, and on any other port `starttls()` raises rather than continuing if the server declines, so the password cannot cross the wire unencrypted. A login refusal is reported with the provider’s own sentence — which says which of the two setup steps was missed — and never with the password.

11.6 The card

Eight cards per A4 sheet at 85×55 mm, the size of a business card, with 2 mm bleed and cut marks. The QR is a version 5-M symbol, which carries the URL with error correction to survive a fold in a pocket. The sheet is rendered from HTML to PDF and was checked by scanning a printed card rather than by trusting the renderer.

11.7 Operational state, and what it cost to verify

The feature shipped across five commits and was verified end to end on the live host: the page reports **open** with the correct closing time, a pass redeemed on the real site created a working viewer account, and the email was accepted by the provider. Four test passes exist on the

deployment and each is a real account with a real end date — which is the point worth making about testing a feature like this. One of them was issued to a university address, and that account is listed in the administration screen; it was disabled, and the cleanup list for after the event includes deleting the test passes, clearing the code, and revoking the application password.

Two defects this feature found in the rest of the system.

Neither was in the pass. A blank value for the closing time in the environment file made *every* request return 500, because a value the settings model refuses raises on every call to the settings accessor — not just in the feature that reads it. The closing time is the one line somebody edits on the morning of an event, so a typo in it must cost the passes rather than the demonstration; three before-validators now turn a blank or garbled value into a disabled feature.

And the closing time stored in SQLite came back three hours out, because SQLite keeps the wall-clock digits and drops the offset, so 23:59+03:00 read back as 23:59 UTC. The countdown on screen said 12:02 for a pass that closed at 09:02. Instants are now converted to UTC before storage, with a round-trip test that fails on the old behaviour.

Chapter 12

Infrastructure and deployment

12.1 Five containers, one host, one file

The stack is declared in one Compose file and can be rebuilt on any host in minutes. That is the proposal’s single-host decision, taken after repeated free-tier account suspensions destroyed a managed environment more than once, and it doubles as the portability argument the project makes anyway.

Service	Image	Notes
timescaledb	timescale/timescaledb:2.17.2-pg16	bound to loopback on a non-default host port; lock table raised to 1024; 3 GB memory limit
mosquitto	eclipse-mosquitto:2.0	loopback only; anonymous access off; per-topic ACL; 8 kB message limit
core	the built image	the application: API, ingester thread, scheduler. One worker, enforced
sim	the same image	different command only, so the simulator and the ingester cannot disagree about the message schema
nginx	nginx:1.27-alpine	static front end, security headers, reverse proxy

Three properties of that file are security decisions rather than configuration.

Nothing has a default password. The database and broker passwords use Compose’s error form, so a host that skipped setup fails to start rather than shipping a broker whose credential is in a file in the repository — and this broker accepts publishes that become readings.

The data directory is mounted read-only into both application containers. Generated output goes to a separate writable mount, which is also where the integrity anchor lives, deliberately outside the database volume.

Published ports are loopback by default. Serving other devices requires setting the bind address *and* the Secure-cookie flag, and the administration screen reports that flag so the pairing cannot silently disagree.

A container’s published port bypasses the host firewall, and that is not a misconfiguration here.

Docker inserts its own rules ahead of the firewall’s input chain, so a **deny incoming** policy does not cover a published port. On the deployed host the cloud network security group is the only thing refusing it — verified from outside, with the group empty, both the application port and SSH refuse. Open one inbound rule there and the dashboard is on the internet whatever the host firewall says. The documentation states this explicitly, because the intuition that two firewalls are two layers is wrong in this particular direction.

12.2 The host

Provider, size	Azure, Standard_D2as_v5 — 2 vCPU, 8 GB, 64 GiB disk
Region	Poland Central
Operating system	Ubuntu 24.04 LTS, Trusted Launch with secure boot and vTPM
Inbound rules	none — the network security group carries zero
Access	a private overlay network; the runner joins it for the length of a job
Public exposure	one HTTPS name, via the overlay provider's ingress
Checkout	<code>/srv/gemp</code> , environment file mode 600

The region and size were forced, not chosen.

West Europe was closed to new subscriptions; the intended smaller size was capacity-restricted in six regions tried; the trial subscription carried over eleven hundred location-level restrictions, which is normal for a trial. Poland Central with this size was the only combination that would launch. The requirement itself was measured rather than guessed — 1.7 GB in use of which the database is 1.46 GB, 3.16 GB of disk growing at 720×, CPU idling low and spiking during a solve — which is what ruled out every 1 GB free tier on the first line.

12.2.1 Reaching it without opening a port

The runner joins the private network for the length of the deploy job and connects over it, so the host never has a public SSH listener. The credential is an OAuth client scoped to a single tag rather than a reusable key: it can be revoked on its own and it does not sit in a repository secret as something that never expires.

The overlay's own SSH feature was disabled, because it is a second door.

Enabling it would put a second path past the forced command into a user who is in the `docker` group — which on any Linux host is root-equivalent. It is switched off on the host and the access-control policy has no SSH block. A privilege boundary with a second entrance is not a boundary.

12.3 Bootstrap: what a fresh host needs

One idempotent script installs the container runtime, creates the deploy user, clones the checkout, generates the environment file at mode 600, generates the broker password file, installs the overlay client and sets the firewall to deny everything inbound except on the overlay interface. It deliberately does *not* open SSH to the internet, does *not* put a registry token on the host, and **does not start anything** — so the first thing the host runs is a release the pipeline built and signed.

Two safety behaviours in it were learned the hard way.

The firewall lockdown is conditional on the overlay actually being up.

Enabling an overlay-only policy while still connected over the public address closes the door you are standing in, and on a cloud virtual machine that means recovering through the serial console. So the script checks for an overlay address; if there is none it leaves port 22 open, says loudly that this is temporary rather than the design, and prints the one command to close it once the new path is confirmed working.

The writable mount needs two owners, and getting it wrong crash-looped the simulator twenty-nine times.

The image runs as uid 10001, which owns the anchor directory *inside* the image — but Compose bind-mounts the host directory over it and a bind mount carries the host’s ownership. After a blanket **chown** to the deploy user, uid 10001 could not write the one directory it exists to write, and the simulator died on a permission error while **restart: always** turned that into a crash loop that *still published readings*. The stack looked alive while the live anomaly ground truth was never written. The fix needs the owner to be the deploy user (because git tracks a file in that directory and a checkout sync has to create it), the group to be the container uid, the directory setgid and group-writable, *and the existing files group-writable too* — appending to a file needs write on the file, not just on the directory. Four conditions, and missing any one of them reproduces the loop.

12.4 Deploying a release

A release is **two things** and both move together: the *image* carries the application, its dependencies and its migrations; the *commit* carries the Compose file, the nginx configuration and the deploy script itself. Shipping only the image was a real hole — change the Compose file and the host keeps running the old one against the new image, silently, forever.

The sequence runs **on the host**, not on the runner, and that is the whole design: a rollback that needs the runner to still be alive is not a rollback.

1. Re-exec from a copy in a temporary directory, because step 3 rewrites this very file and bash reads a script incrementally rather than all at once — a file that changes mid-run makes the shell resume at a byte offset into different text.
2. Resolve the reference to a digest. A tag is a pointer and pointers move.
3. Sync the checkout to the release commit, fetching by commit rather than by branch, because deploying “whatever the branch points at now” is how a host ends up running a commit nobody chose.
4. Record what is running: image *and* commit.
5. Migrate, from inside the new image. A failure here aborts before the running containers are touched — the cheapest possible failure, because nothing has changed.
6. Swap the application containers, then restart nginx.
7. Health-check, then smoke-test.
8. On failure: restore the recorded image *and* commit, verify that, exit non-zero.

nginx must be restarted, and the reason is invisible.

It resolves the upstream hostname once at startup, so a recreated application container gets a new address and every request answers 502 while the process list reports everything healthy. On the day, that is a dead demonstration with no error message pointing anywhere near the cause.

Migrations are forward-only, and that is what makes rollback safe.

There is no automatic downgrade and there must not be: a downgrade that drops a column destroys the data in it, and the rollback would become the thing that loses the readings. So a schema change has to be compatible with the release *before* it — add columns, do not rename them, drop the old ones a release later. That expand-then-contract discipline is the contract that lets the previous image run against the new schema.

12.4.1 What counts as “it works”

The health endpoint proves the process is up and the database answers. It does not prove the release is any good — a build with a broken front end, a stale upstream, or an authentication middleware that fell open passes it without noticing. So three more checks run, each failing for a different real reason:

Check	Catches
/ serves the application shell	broken nginx, missing front-end mount
/api/v1/auth/session says nobody is signed in	stale upstream 502, unreachable application
/api/v1/meta returns 401	authentication failing open

The third is the one worth having. An authentication bug that lets everyone in makes every other check pass: everything works, and works for everyone.

12.4.2 The privilege boundary

The deploy key is restricted to a forced command. It does not run what the caller asked for — it reads the request, validates it hard, and calls the deploy script itself:

- the image reference is rejected if it contains any shell metacharacter, must match a strict pattern, and must come from the expected registry;
- the commit must be a hexadecimal object name;
- the reseed argument must be a whole number of months inside a range;
- only three scripts may be run, matched on the script *field* rather than anywhere in the request — a wildcard match would also match an image tag containing the string;
- no pseudo-terminal, no port forwarding, no agent forwarding, no user run-control file.

This restriction is not defence in depth — it *is* the boundary.

The deploy user must be in the **docker** group, which on any Linux host is equivalent to root: the daemon runs as root and will bind-mount anything. Without the forced command the key is a root shell with extra steps. That is why a proposal to add a fourth script behind it, to save a click during this project, was refused: widening the one restriction that constitutes the boundary to save an operation that a person can do another way is the wrong trade.

The registry credential is handled with the same care. It arrives over standard input — never as an argument, which would be visible in the process list for as long as the pull takes, and never as a file, because the bootstrap promises not to put a registry token on the host. It is a short-lived token minted per job, and the script logs out on every exit path.

12.5 Rebuilding the history

Two things drift on a long-running deployment: the simulator advances data time faster than the clock, so history ends wherever that ratchet took it; and a portfolio replaced in the repository does not reach the database until something re-imports it. One operation fixes both, and it is a separate workflow from deploy rather than a flag on it — a deploy that could wipe the readings depending on an input is a deploy nobody can run without reading the form twice.

Both writers stop, not just the simulator, and finding that out cost an afternoon.

The application runs the ingest consumer, which holds each chain’s head in memory and mirrors it to the external anchor once a minute. Stopping only the simulator leaves it running while the table is emptied and rebuilt underneath it, and the two ends of the integrity check then describe different worlds: measured on this host after a reseed, the anchor claimed sequence 69 066 while the table stopped at 69 063, and the integrity claim failed with “the tail has been deleted” — the alarm firing correctly at a discrepancy the operation itself had created. The API is therefore down for the length of the seed, which is the honest trade: an operation that rebuilds the entire reading history is not one to serve requests through.

The operation also states what it destroys, because the script cannot ask: every stored reading, and with it every anomaly and integrity checkpoint that referred to one. What survives is deliberate — accounts, sessions, the audit log, stored allocations and the catalogue. A stored allocation whose readings are gone is still a record of what was decided, and still verifies against its own input hash.

12.6 Backups, and the drill that found a real defect

Everything the platform demonstrates lives in one volume. Three things are captured, because restoring any two of them is not a restore: the database in a custom-format dump, the anchor file that proves the chain was not rebuilt along with the rows, and the environment file holding the signing key — readings restored without it can never be verified again, and the integrity feature becomes decoration.

The script refuses to keep a dump under 4 kB, because a zero-length dump is a failure that exited zero, and it prunes to the last seven so that the backup filling the disk does not become the outage.

A plain restore silently lost a constraint, and only an assertion caught it.

The first real drill restored all 7.5 million rows and reported success — while failing to recreate the foreign key from readings to buildings, which is the constraint the ingester depends on to refuse readings for unknown buildings. A hypertable is a parent table plus hundreds of chunks wired together by the extension’s own catalogue, and restoring that with a plain restore lets the extension’s triggers fire while chunks are still arriving. The procedure now brackets the restore with the extension’s documented pre- and post-restore calls, and **asserts** on both the error count and the presence of that constraint rather than printing “OK” regardless. A restore that drops constraints produces a database that looks right and behaves differently. Second run: zero errors, foreign key present.

The rehearsal form restores into a scratch database, counts rows *and* constraints, and drops it — because restoring over the live database to find out whether the backup works destroys the thing you were protecting if it does not. One caveat is stated rather than glossed: the default destination is on the same disk as the database, which covers corruption and a bad restore but not a disk failure or a lost laptop.

12.7 The drill that needs no server

The most valuable piece of this chapter’s infrastructure owns no hardware. A workflow makes the throwaway runner *be* the host: it builds the release and a second image that starts and then dies, brings up the database and broker as a real host would already have them, deploys the good release, verifies the schema was actually migrated rather than assumed, then deploys the

broken one and **requires it to fail**. It then checks that the previous release is serving again by image identity, that a failed deploy did not move the rollback pointer, and that a backup restores.

A rollback path is only ever exercised on the worst day unless something exercises it on purpose — and this one found a real bug.

The simulator waits on the application being healthy, so a release that starts and then fails its container health check makes Compose exit non-zero with “dependency failed to start”. Under `set -e` that killed the deploy script on the spot — *before* the health check, and therefore before the rollback. The rollback path was unreachable for the single most likely way a deploy goes wrong, and nothing but a deliberate failure would ever have shown it.

Chapter 13

The pipeline

13.1 One workflow, three jobs, ordered by how fast they fail

Job	Budget	Gate
gates	15 min	lint; the test suite with a coverage floor; the data contract; the claims harness; citation status reported separately
secure code review	25 min	seven scanners, every report normalised, one verdict
build and sign	20 min	multi-architecture build, pushed, signed keylessly, SBOM attested — and only from the default branch

The ordering is deliberate: the cheap checks that fail most often run first, and the build happens only if both preceding jobs passed, because there is no reason to push an image nobody may deploy.

Permissions are read-only by default and each job asks for exactly what it needs, so a compromised action in one job cannot write to the repository from another. The build job is the only one with package-write and identity-token permissions, and it is gated on being a push to the default branch — a pull request from a fork must not be able to push an image, and a green pull request is not a release.

13.2 The security policy: block on anything

Any finding from any scanner fails the workflow, and a failed workflow cannot deploy because the deploy is triggered *by* it and reads its conclusion. Pushing straight to the default branch does not skip the review.

Tool	Looks at
ruff	style and dead-import drift (a gate, not a security tool)
bandit	Python patterns: subprocess, weak hashes, assertions in production paths
semgrep	deeper static analysis: OWASP top ten, injection, secrets in code
pip-audit	advisories against the declared dependencies
gitleaks	secrets in the tree <i>and in history</i> — a deleted secret was still published
hadolint	the container build file
trivy	filesystem, infrastructure-as-code misconfiguration, and the built image

Every scanner runs with continue-on-error, and that is what makes the gate trustworthy rather than weaker.

A scanner that crashed must not be indistinguishable from a scanner that found nothing. So no scanner is allowed to fail the job; instead a single gate script reads every report afterwards and is the only step permitted to fail. It treats an **unreadable report as a failure**, not as an absence of findings — a tool that died halfway through writes truncated output, and treating that as clean is how a broken gate reports success for a month. This has a trap attached, and it caught the team once: a green tick on the bandit step means nothing. The gate’s summary is the thing to read.

13.2.1 The one thing that does not block

“Block on anything” means anything that can be acted on. The base image carries roughly 120 operating-system advisories with no patched version published *anywhere* — not “not upgraded yet”, but nothing to upgrade to. Blocking on those makes red the permanent state of the build, and a gate that is always red is one everybody learns to click past. The exception register cannot absorb them either: 120 entries all saying “upstream has not shipped a fix” is a rubber stamp with a date on it.

So the two scans feeding the gate ignore unfixed advisories, and the build file applies an operating-system upgrade at build time — which means a fix becomes *this build’s problem* the moment one is published. That combination took the image from 69 fixable findings to zero. Nothing is hidden: a third scan records every finding including the unfixable ones into a separate directory, uploaded and kept as an artefact, which the gate does not read. When upstream ships a fix, the finding moves from that list into the blocking one on its own.

The final image also ships no package manager. Removing `pip`, `setuptools` and `wheel` was the only way to clear the last three findings: the package manager vendors its own copies of two libraries, pinned by its release rather than by anything this project can pass to it. It is also simply correct for a runtime image — nothing installs packages after that line.

13.2.2 Getting past a finding

One file, reviewed in a pull request like any other change. Every entry names the finding, who accepted it, why, and **the date the acceptance runs out**. An expired entry fails the build on its own, whether or not the finding is still there. The register is currently empty.

The expiry is the mechanism, not a formality.

An exception without one is a policy change made by whoever was on shift, and it outlives every memory of why it was made. The realistic alternative is not a carefully maintained permanent exception — it is somebody adding a shell short-circuit to a workflow step at eleven at night, after which the gate is decorative. An honest way out is what keeps a strict gate strict.

In-source suppressions exist for the handful of findings that are architectural facts rather than backlog items — the module that exists to run a subprocess, the fixed-URL requests — and they are marked at the line with the reasoning at the import. They carry no expiry, so the gate prints every one of them in every summary rather than letting them disappear into a scanner’s own bookkeeping.

13.3 Supply chain

The image is built for two architectures and signed **keylessly**: the build job exchanges a short-lived identity token for a certificate, so no private key exists to leak, rotate or forget. Signing is

by *digest*, never by tag, because a tag can be moved to point at something else after it is signed. A software bill of materials is generated and attested alongside it.

The deploy job then verifies that signature *before anything touches the host*, against a certificate identity that must be this repository’s own workflow. An unverifiable image never reaches the server — that is the last point at which stopping costs nothing.

Two pipeline bugs worth recording, because both were invisible.

An action referenced as @0.28.0 where the published tag is v0.28.0 meant the entire security job died in three seconds at job setup, on every commit and every pull request, having **never once executed a single scanner**. A missing tag kills a whole job before any step runs, and in the interface it reads as a security failure while being nothing of the kind.

And the registry path: the repository name preserves the case it was created with, while an image reference must be lower case. The metadata action lower-cases silently, so the *push* worked and the signature step failed — and the certificate identity is a URL that must *not* be lower-cased, so the two had to be split apart under separate names.

13.4 Deploy, and the human gate

Deployment is triggered by a successful run of the main workflow on the default branch, and additionally requires a repository variable to be set — which exists because until the server existed, every green run reached the deploy job, failed at the connection step and opened an issue reporting a broken deploy when the truth was that the host had not been built yet. Manual dispatch ignores the variable, so it can never block a deploy somebody asked for.

Two environments, each with its own secrets and its own approval rule. Manual runs default to staging, so the careless click is the safe one. Giving the production environment a required reviewer makes every deploy there wait for approval; leaving it open makes it one click.

A failed deploy opens an issue labelled for it, with the image, the commit and a link to the run. The host has already rolled itself back by then, which is exactly why it needs saying out loud: the site is up and serving the *previous* release, and without a notification the next thing anybody notices is demonstrating yesterday’s build. An issue rather than a chat webhook — no third party, no secret to configure, and it survives the demonstration network.

13.5 The same script for a person and for the pipeline

Every operation has one name, and the pipeline and a person run the *same* one: `make check`, `make security`, `make deploy REF=...`, `make rollback`, `make released`, `make backup`, `make backup-verify`, `make restore DIR=...`

A deploy only the pipeline knows how to perform is a deploy nobody can do at eight in the morning on the day of a demonstration.

The same reasoning applies to the local security target: it runs the identical scanners with the identical flags, from container images where a native binary is not available on a developer machine. A local review running a smaller rule set than the pipeline is worse than no local review, because it reports success on a push that is about to go red.

13.6 Known gaps, stated rather than discovered

- **Actions are pinned to major tags, not commit digests.** Digest pinning is stricter and is the right end state; a wrong digest is a workflow that cannot run at all, so it is a

deliberate follow-up rather than a value guessed in a hurry. The dependency bot watches the tags weekly — though see the next point.

- **Repository-level alerting is off.** Dependency alerts, secret scanning and code scanning are disabled, so the pipeline’s upload of findings to the security view silently does nothing. The in-pipeline scanners still block, so this is missing *notification and history* rather than missing *enforcement* — but it is the reason the dependency bot’s tag-watching does not currently reach anybody.
- **The fifteen integration tests are not a gate anywhere.** The workflow starts no service containers, so they skip there exactly as they skip on a laptop with nothing running. “533 passed” in a log means 533 of 548, and it is worth knowing which fifteen are missing. Adding two service containers is the fix; it was not done close to a deadline because a newly-running integration test that fails also blocks every deploy.
- **Branch protection is weaker than the policy assumes.** The default branch requires no status check and no review, so “block on anything” governs what *deploys* rather than what *merges*. This is finding S-1 of Chapter 10, and freezing auto-deploy closed its live edge.
- **The deploy path needs the network.** The runtime guarantee is unaffected — the stack still runs with the cable out — but shipping a release to an air-gapped host means saving and loading an image by hand. The script handles that case; nothing automates the transfer.
- **The signing key is on the host.** The deploy refuses to run without it and warns if its permissions are wrong, but nothing provisions or rotates it, and regenerating it invalidates every existing signature. That is a deliberate property, and it means the key must be backed up with the data it signs.

Chapter 14

Testing

14.1 What the suite is and is not

548 tests collected in seven seconds, from 472 test functions across 27 files, expanded by parametrisation. Fifteen are integration tests that skip themselves when the broker and database are not running, so **533 run anywhere** — on a laptop with nothing up, and in continuous integration, which starts no service containers.

Table 14.1: Test functions per file. The collected count is higher because fourteen parametrised decorators expand some functions into many cases — `test_prune.py` is 8 functions and 55 cases.

File	Fns	File	Fns
<code>test_api.py</code>	52	<code>test_harness.py</code>	15
<code>test_auth.py</code>	48	<code>test_profiles.py</code>	13
<code>test_judge_pass.py</code>	35	<code>test_live_anomaly.py</code>	13
<code>test_web.py</code>	33	<code>test_security_gate.py</code>	12
<code>test_ml.py</code>	31	<code>test_savings.py</code>	12
<code>test_evaluation.py</code>	30	<code>test_integration_pg.py</code>	12 [†]
<code>test_optimize.py</code>	22	<code>test_bootstrap.py</code>	11
<code>test_catalog.py</code>	20	<code>test_webhook.py</code>	10
<code>test_integrity.py</code>	18	<code>test_units_and_e2e.py</code>	10
<code>test_lifecycle.py</code>	16	<code>test_replay.py</code>	9
<code>test_ingest.py</code>	16	<code>test_prune.py</code>	8
<code>test_portfolio_import.py</code>	7	<code>test_grafana_archive.py</code>	7
<code>test_ci_workflow.py</code>	6	<code>test_sim_clock.py</code>	3
<code>test_integration_mqtt.py</code>	3 [†]		

[†] integration-marked: skips itself without the stack. 15 functions in total.

14.2 Fixtures are built in code, never read from the data files

A test that breaks when a cost estimate is revised is a test that gets deleted under deadline pressure.

The values in the catalogue are the other author's to change. If the suite asserted on them, every citation Nada sourced would turn the build red and the pressure to delete the assertion would arrive at exactly the wrong moment. So fixtures are constructed in `conftest.py` with their own parameters, and the data files get *structural* validation instead — shares summing to one, cross-references resolving, citations present. The two kinds of check are kept apart on purpose.

14.3 The suite enforces what the database enforces

SQLite ignores foreign keys unless a pragma is set, which makes the contract tests a weaker check than the database they stand in for. That is not hypothetical: a run row and its allocations were being inserted in the wrong order, PostgreSQL rejected the transaction, and the API returned an identifier for a run that never existed — while the SQLite-backed tests passed happily throughout. The pragma is enabled on every connection now.

A suite that accepts what production rejects is worse than no suite, because it converts an outage into a surprise.

The same reasoning drives the integration tests that do exist: they run the *real* consumer against a temporary database, which is how a PostgreSQL-only interval literal in the detector’s warm-up was found. That function had taken an injectable session factory specifically so it could run against SQLite, and could not — the contradiction sat unnoticed because nothing ran the real consumer end to end until Phase 4.

Property-based tests are used where a round trip or an invariant is the thing worth pinning rather than an example — the money-unit round trip of **F12** among them.

14.4 The four gates, runnable locally

One script runs exactly what the pipeline runs, in the same order, reading the same list: lint; the suite with a coverage floor of 80%; the data contract; and the claims harness. Gates run to completion rather than stopping at the first failure, because finding out that three things are broken should take one run rather than three.

The coverage floor sits deliberately below the measured figure — 83% without the container stack, 84% with it. It is there to catch a module losing its tests, not to be a target anybody optimises. Command-line entry points and the simulator are excluded, because writing tests that assert on argument parsing would raise the number without raising the confidence.

14.5 Tests that exist because of a specific defect

A proportion of this suite is regression pressure rather than specification, and those are the ones worth naming.

Test	The defect it pins
deny-by-default route walk	a route declaring no role. It descends into included routers, because the framework keeps one as a single wrapper entry rather than flattening its children — reading only the top level reports that every auth and dashboard endpoint does not exist
the public-path set is exactly this set	a path being added to the public list by accident
pruning changes no solver’s result	the safety argument for dominance pruning, checked across every solver, objective and budget
the solver cap does not assert the hardware	a pin that held on a 12-core laptop and failed on a 2-core runner, where two limits collapse to one
the portfolio fixture is versioned	a fresh checkout regenerating a <i>different</i> portfolio with a different generator, so the harness would re-measure the paper’s claims against a system the paper does not describe — silently, and passing
the archived dashboards are an archive	a passing test file named after provisioning, implying a service that does not run
a sixth container appears	the five-container claim

One test's first line is a sentence rather than an assertion.

The archived dashboard provisioning is checked as something that *would* provision cleanly, not as a service. The file's docstring opens by saying the product is not deployed, because the previous filename implied the opposite to anyone scanning the directory. A test suite is also documentation, and a misleading name in it is a defect.

Chapter 15

The evaluation harness

15.1 A claim is not a test

| The distinction is the whole reason this exists.

| A **test** pins behaviour that must never change. A **claim** re-measures something already asserted in prose, so that the prose can be corrected when the measurement moves. The two want opposite things from a failure: a failing test means the code is wrong; a failing claim may mean the *sentence* is wrong. Conflating them produces either a suite nobody may change or a document nobody checks.

`python -m gemp.evaluate` re-solves the instance grid, re-measures fourteen numbered claims — nineteen when a database and a completed refit are available — writes them to CSV, renders them in the application’s own evidence view, and **exits non-zero when a claim stops holding**. It runs in continuous integration on every commit.

15.2 How the grid is built

Expanding the candidate set is the expensive part and does not depend on the budget, the objective or the cap — so it happens once and every cell of the grid reuses it. The full sweep is $20 \text{ budgets} \times 3 \text{ objectives} \times 2 \text{ cap settings} \times 4 \text{ solvers} = 480 \text{ solves}$, measured at about twenty seconds.

Rows carry the funded candidate *keys* as well as the totals, because two claims ask *which* options were funded rather than how much benefit they delivered, and re-solving to answer that would let the two answers drift apart.

| The reporting range is a function of the portfolio, not a constant.

| It was 2–40 MEGP when the fixture was mostly apartment blocks. The portfolio is public buildings now and their retrofits cost proportionally more: 25 M funds twelve of the fifty, 100 M funds thirty-six, 200 M funds all of them. A sweep stopping at 40 M would report the bottom fifth of the curve and miss the point where the solvers converge — which is exactly where several claims are measured. The first version of the quick grid topped out below the point plain greedy saturates, so two claims reported failures that were artefacts of the grid rather than findings about the solvers.

A reduced four-budget grid exists for the gate, chosen to span the whole decision range rather than to be small. There is one important consequence, and it is documented at the job that runs it: the quick grid prints different figures for the same claims — 72 instances where the full sweep says 360, a median of +1.43% where it says +1.25%. Both are true; only one of them is the number on the poster, so the button in the interface runs the *full* sweep and the quick grid stays in the pipeline.

15.3 Three properties that make the output worth reading

Claims are gated and can fail. Every threshold was fixed before measurement. They are loose on purpose — a gate tight enough to trip on a catalogue revision would be deleted the first time a citation moved a cost by ten per cent.

Insufficient evidence reports skip, never pass. A claim needing the database skips when no refit has run rather than reporting the last value it saw. A median over fewer than three instances skips, because a median over two instances is not a median — an artefact of the grid presented as a finding about the solver is precisely what this harness exists to prevent.

Solvers are paired on identical instances, keyed by (budget, objective, cap) rather than by list position, so a changed grid cannot silently compare a capped run against an uncapped one.

15.4 What it caught

This is the part that justifies the harness, and all three were things the project had *already written down as results*.

1. **The greedy saturation defect.** The headline compared CP-SAT against a baseline that stops spending, and was therefore measuring that baseline’s ceiling. A stronger baseline was written and every headline requoted against it (§6.3).
2. **A mis-specified bundling claim.** The first version demanded that every bundle save strictly less than the sum of its parts, and 509 of 1 114 bundles failed it. The *claim* was wrong, not the model: lighting and zone control act on different end uses and generation is not a load reduction, so there is no interaction term for them to lose. The claim split in two — bundles sharing an end use must be strictly sub-additive, and no bundle at all may exceed the sum of its parts — and both now hold at 662 of 662.
3. **Two anomaly scoring artefacts,** each of which had produced a number the paper could have quoted: a forecaster predicting negative load into a detector that divides by the expectation, and a simulator that had filled the ground truth with 1 762 phantom events out of 2 212.

It also caught a false alarm in the one feature whose purpose is to be believed.

The first run against the live stack reported an integrity claim failing on a building nobody had touched — because verification read the rows before the anchor, and ingestion had checkpointed past the snapshot in between. The ordering was inverted so the check is one-sided in the safe direction. A harness that only ever confirms is a marketing document; this one has contradicted the project four times.

15.5 Exit status is the product

Zero means every claim the paper makes still holds. One means a sentence somewhere needs rewriting. A claim documented as open would report FAIL and be counted separately — because a gate that is permanently red teaches everybody to ignore the gate — and there are none open today: the last one, the uncited catalogue, closed on 1 September and is now blocking.

The evidence screen has a property worth knowing before a demonstration.

The harness writes its output to a file inside the application container, and every deploy replaces that container. So the result of pressing the button does not survive the next release, and the screen honestly reports that no measurement has been taken on this deployment rather than showing a stale table. The operational consequence is an ordering: press it *after* the last deploy, not before.

Chapter 16

Engineering chronology

16.1 Shape of the work

132 commits between 14 August and 12 September 2026, adding 59 792 lines and removing 11 273 across 844 file-changes. The distribution is not even, and the unevenness is the story.

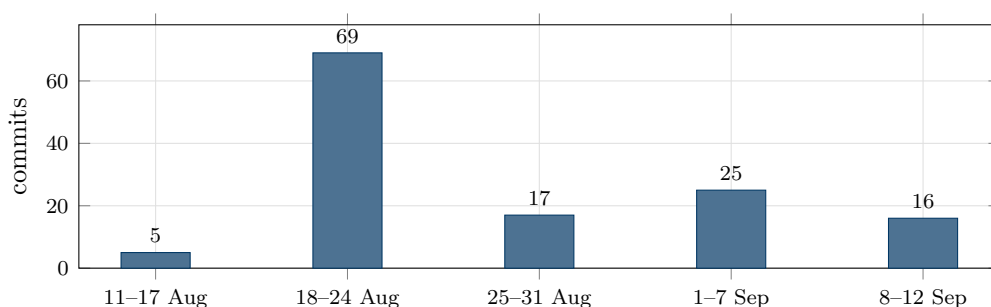


Figure 16.1: Commits per week. The spike is the week the whole platform was built — phases 0 to 4 and the identity layer — against the original 31 August deadline. The later weeks are the deadline moving to 9 September and being spent on the things a demonstration actually needs: citations, a phone layout, Arabic, the time-of-use layer, and the evidence the claims rest on.

16.2 Phase by phase

Table 16.1: The phases, with the commits that opened and closed each.

Phase	Dates	What landed
0	14–15 Aug	Domain model and budget-constrained optimiser, against CSV fixtures with no infrastructure (a05a507). Real OpenStreetMap footprints and dominance pruning (46d559b). <i>Exit gate: a real allocation over fifty fixture buildings, shippable with nothing behind it.</i>
1	15–17 Aug	Integrity chain, simulator, infrastructure; the MQTT node and the end-to-end signed write path; the HTTP API, repository and services; the full stack running (787ac2e → 74f1209). Dataset replay and candidate materialisation closed the gaps (85afeda).
2	17 Aug	Forecasting, anomaly detection and threshold calibration (5675410), then the first real correction: anomaly F1 from 0.24 to 0.62 (508593d). Scheduler, streaming detection and cache invalidation (dff9a35).

Phase	Dates	What landed
3	17 Aug	Map interface, controls, dashboards (9032a9b), and an audit of them the same day (8afb13).
4	17–18 Aug	The evaluation harness <i>and the baseline defect it found</i> (69069d6) — the single most consequential commit in the project. Then a cascade of things the harness and a code review exposed: the anomaly evaluation scoring its own correct detections as false alarms; three 500s on the demonstration path; the tamper-evidence that was never actually checked; the ingester that could not run against SQLite; a false truncation alarm found by the claim meant to prove the opposite.
—	17–19 Aug	Identity, roles and sessions — the platform stops being open by default (c0bf4a1); dashboards move in-app and the second product leaves the stack (4e43842); the interface rebuilt on the design handoff (8317fd5); the pipeline (a1a29c2).
—	20 Aug	Ten commits fixing a pipeline that had never once worked. §16.4.
—	25–26 Aug	The login boundary hardened after a penetration test; metered load shapes reach the funding decision (471ccce).
—	27 Aug–1 Sep	Nine screens plus a landing page rebuilt; Arabic across the interface; a ceiling on the data clock; fifty public buildings OpenStreetMap actually names (18ab17b); a reseed path; every catalogue rate cited (704bacf).
5	3–12 Sep	The phone; every parameter cited and four corrected (a0c3ef9); the evidence job measuring claims that need the database (ff0ccae); counting one claim once (f178884); the Judge Pass (3d01d67 → 5ebda81); the security review’s one code fix (43ef04f).

16.3 The week the deadline moved

The original deadline was 31 August. It moved to 9 September, which bought nine days, and the decision about how to spend them is worth recording because the obvious answer was wrong.

The obvious answer is polish. The answer taken was to close the one *structural* gap: until then the optimiser decided from a single scalar per building against one constant emission factor, so hundreds of thousands of signed readings changed **nothing** about which building got funded. The sharpest question a judge could ask — “you ingest all this data; does any of it change what you fund?” — had the honest answer *no*.

The feature that closed it is the time-of-use layer of §5.5, and the plan for it named the day that would get skipped under pressure:

Day 7 is the one that gets skipped under pressure. Do not skip it — the feature without re-running the harness leaves a system whose numbers disagree with its own write-up.

It was not skipped.

16.4 Ten commits on 20 August

The pipeline was committed on 19 August and had never worked. Every failure was silent in a different way, and the sequence is instructive because each fix revealed the next.

Commit	What was wrong
fedef31	An action referenced by a tag that does not exist. Every reference is resolved before any step runs, so the security job died in three seconds at setup on every commit and pull request — having never executed a single scanner.
6abc3a3	The registry path kept the repository’s capitalisation. The metadata step lower-cases silently, so the push worked and signing failed — and the certificate identity is a URL that must <i>not</i> be lower-cased.
7049b87	The drill never generated the broker password file, so the runtime created a <i>directory</i> at that path and the broker exited before its first health check.
0c48d7e	The forced command took the <i>last</i> token of the request as the image reference — which is the commit — so every deploy was refused, and the commit was never forwarded.
b808274, 5bc246d, 924aed5, a685e57	Four commits to get one directory’s ownership right. The simulator crash-looped twenty-nine times, publishing readings and dying, silently losing the live ground truth.
7e483c6	The registry token was read and thrown away: the read returns non-zero at end-of-input without a line terminator <i>even though it filled the variable</i> , and the handler cleared it on that “failure”.

Every one of these produced a green-looking system.

A job that dies at setup reads as a security failure in the interface while being nothing of the kind. A crash-looping container still published readings, so the stack looked alive. A deploy refused with exit 126 looks like a permissions problem. The common shape is that none of them announced what was actually wrong, which is why the drill of §12 — deploy something deliberately broken and require it to fail — earns its place: succeeding proves less than failing on purpose.

16.5 How the work was handed over

Five session handoff documents were written — 18, 19, 27 and 28 August, and 12 September — each replacing the last, each carrying standing rules, current state, what is left and the traps learned. They are deliberately *not* in the repository: the repository holds the platform, and the engineering record is kept beside it.

That separation is itself a decision with a cost. It means this report is the first place the two are joined.

One rule in those documents outlived every session and is worth stating.

When a measurement contradicts an earlier claim, the claim gets corrected in writing rather than quietly dropped. Several numbers in the project’s own documentation are corrections of earlier ones and say so. That is why this report can quote both the superseded figure and the current one in half a dozen places — the superseded ones were never deleted, so they can still be compared.

Chapter 17

Defects and lessons

This is the chapter the authors would hand a successor first. Every entry cost real time, and the ones worth recording share a property: **the system kept working, or appeared to, while being wrong.**

17.1 The governing pattern

Code that is written but never called looks exactly like code that works.

The truncation-detection function was implemented and unit-tested in Phase 1 and called by *nothing* until Phase 4: the endpoint took checkpoint arguments and no caller ever passed them, so the integrity response reported “unknown” on every request the demonstration ever served. Tamper detection for *modified* rows was real; for *deleted* ones it was absent — and deletion is the easier attack. Unit tests do not notice an unwired capability. Only a claim measured against the running system does.

The same shape appeared three more times: a replay path for real meter data built in Phase 1, documented nowhere and referenced by no compose file; an acknowledge column that two queries filtered on and nothing could set, so an inbox of eleven thousand alerts only ever grew; and an endpoint the interface never called. The lesson the project drew is *verify against the running system, not against intent* — most of the defects found in Phase 4 were things that were implemented, tested, and never actually called.

17.2 Time, and a clock that is not the wall clock

The simulator replays at 720×, so data time and wall time diverge immediately. Every rolling window is therefore evaluated in *data* time, anchored on the newest stored reading. A window anchored on the current time silently returns empty.

Symptom	Cause
The simulator publishes and nothing is stored	Unable to reach the API at startup, it fell back to “now” — which under accelerated replay is over a <i>year behind</i> the stored data. Every reading it published was a duplicate that the insert discarded, while it went on recording the faults it thought it was injecting. 1 762 of 2 212 ground-truth events described readings that were never written, reading as recall 0.375 for a detector whose recall is 0.84. It now retries and then refuses to start.
The data clock reads 2040	Acceleration with nothing above it is a one-way ratchet: the simulator resumes from the newest stored reading, so a restart continues the climb. Ten days of uptime put readings twenty years ahead and grew the database to 35.7 million rows. A clamp now spends the acceleration only on catching <i>up</i> .
The aggregate tail is a day behind	A continuous aggregate’s refresh policy is scheduled in <i>wall</i> time while the data it materialises advances in data time. At 720× a five-minute schedule is sixty hours of data. That is the design working; comparing the newest buckets against the raw table measures refresh timing rather than correctness.
The forecast panel draws one line	The data clock advances at 720× while the refit runs on a wall-clock schedule, so within a few hours of wall time the window contains readings and no forecast. The chart is right; what it is drawing has aged out.

17.3 Databases

- **Never use localhost for the database from the host.** The container publishes on IPv4; Windows resolves `localhost` to the IPv6 loopback first and stalls on a timeout. Measured: 130.3 seconds against 0.2. It presents as “the database is slow”, not as an error.
- **Never read the hypertable per building with a bound parameter.** On 253 chunks the planner cannot exclude chunks for a value it has not seen: 130 s for 6 694 rows against 1.7 s for all 334 702 unparameterised. Read the whole portfolio once.
- **An unqualified count is not free.** `SELECT count(*) FROM reading` measured 6.1 seconds against the deployment — 35.7 million rows across 250 chunks, which the planner turns into a 250-way append and then compiles into 2 252 functions. Two screens paid that on every load, and it was 98 % of their wait. Asking the extension for its own estimate returned in 13.6 ms, about 450 times faster, running 4.6% low — so the screen says “about” and the caller is told it is an estimate.
- **PostgreSQL-only syntax breaks the SQLite contract tests.** Interval literals in particular; compute cutoffs in Python and pass them as parameters.
- **A lazily-built engine needs a *reentrant* lock.** The session-maker holds the lock and calls the engine getter, which takes it again — a plain lock deadlocks the first thread that builds both, which is every start-up. The application logged “startup complete” and hung on the first request that touched the database.

17.4 Containers

- **nginx caches the address of its upstream for the life of the process.** Recreating the application container leaves every request answering 502 while the process list reports healthy.

- **A container that writes a file needs a mount for it, and nothing says so when it does not.** The simulator wrote the ground truth for every fault it injected *inside* the container, because only the other service mounted the directory. It ran for months of data time with no error anywhere, and the loss became visible only as an anomaly precision of 0.26 that should have been 0.52.
- **Building one service does not rebuild the other** — they are separate images from the same file.
- **A container restarted is not a container recreated.** The broker restart-looped on a missing access-control file purely because it predated the new mount.
- **A published port bypasses the host firewall**, because the runtime’s rules sit ahead of it. The cloud firewall is the only thing refusing it.

17.5 The browser

- **A policy of default-src ‘self’ discards inline styles silently.** Three style attributes sat in the map module for weeks, doing nothing; the modal cards never had the layout their markup asked for.
- **Any author rule setting display beats the browser’s [hidden] rule.** Both modals rendered on page load and the map opened behind a full-screen overlay reading “Solving...”, with nothing solving.
- **A shorthand beats a longhand declared earlier at the same specificity**, and the symptom looks like a layout bug rather than a cascade one. A blanket padding silently zeroed children’s own vertical padding; a `margin: 0` meant only to clear heading margins killed a band’s auto-centring and left a heading 195px from the grid it belonged to.
- **Bare, generic class names collide, and the symptom is never obviously CSS.** A status strip carrying a class that was also a grid component made *every* status message in the product render stacked instead of on one line.
- **A rem font size on the root element compounds against itself.** The whole type scale came out 12.5% smaller than designed — body text at 12.25px where the tokens promised 14.
- **transform-box: view-box does not resolve against a nested SVG’s own viewport.** A rotor’s origin landed 55px from the hub and the blades orbited for weeks.
- **An animation that writes content can leave a *wrong value* on screen.** A count-up parked on zero for a third of a second under a throttled renderer — a false number, on a page whose entire argument is that its numbers are checkable.

17.6 Measurement

These are the ones that changed published numbers, and they are the most important entries in the chapter.

Defect	What it did to a reported figure
Mixed units in scoring	Recall counted over events, precision over flagged <i>hours</i> . A ten-hour fault flagged for six scores as six predictions rather than one alert. Reported precision ≈ 0.19 for a detector whose episode precision is far higher.
Scoring past the ground truth	Data keeps arriving; the truth file does not. Every correctly detected later fault counted as a false alarm: precision read 0.26 where it was 0.50.
A negative forecast	The detector divides by the expectation, so an expectation near zero turned an ordinary 1 kW reading into a robust score of 15 319 — ranked above every real fault in the portfolio.
A generator shortcut	Applying the monthly seasonal factor as a step made load jump 17% at midnight on the first of each month. Nothing predicts that jump because nothing causes it, so the forecaster mispredicted every month boundary and the detector duly reported each one. A modelling shortcut in the <i>simulator</i> was manufacturing false positives in the detector.
Two calendars	When the simulator knew about holidays and the feature builder did not, the model mispredicted holiday load by $\approx 70\%$: the five worst days in the evaluation were all public holidays and the top twenty dates were 58% of every false alarm.
A test key in the process	Test modules are imported before any test runs, so one setting an environment variable applies to the whole session — and environment beats the configuration file. It presented as an intact production chain reporting “modified at seq 0”.

17.7 Process

A green suite proves nothing about what a control *says*.

Four user-visible defects shipped past every gate because no test read the text of a control: the default language rendered raw translation keys on every screen, three keys were in neither table, the right-to-left work was written against a stylesheet that had no logical properties, and the print path printed the navigation rail. The suite was green throughout. The regression test written afterwards asserts on what a control says, and was verified to fail with the offending keys named — because a regression test nobody has seen fail is a hypothesis.

- **Three of the ingestion fixes were fixes to the previous fix** — retry-everything, then wedged on constraint violations, then dropped whole batches, then isolate the offending rows. A constraint violation is deterministic and retrying it is retrying forever; transient and deterministic failures need different handling. The suite passed clean at every one of those steps; what caught them was deploying and reading the health endpoint.
- **Check the path before declaring a root cause.** A probe of the wrong session endpoint returned 401 and looked like a routing bug; the real endpoint returns 200 with a payload saying nobody is signed in. One wrong URL produced a confident and entirely fictional diagnosis.
- **The mock can be kinder than the API.** A local harness answered the login endpoint with an empty success, so the one flow that could not be exercised against it was the one being debugged. Know which paths a harness cannot prove.
- **Two sessions editing one tree caused real friction** — uncommitted work sat while another session committed around it, and one commit swept in changes its message did

not describe.

- **A test can assert the hardware.** A pin comparing two concurrency limits held on a twelve-core laptop and failed on a two-core runner, where both collapse to one.

17.8 The shape of a good failure

Three properties recur in the defects that were cheap to fix and expensive to find, and they are worth stating as design guidance rather than as history.

Fail loudly at the boundary rather than quietly in the middle. The application refuses to start under more than one worker; the simulator refuses to start rather than guessing a resume point; the deploy refuses to run without a signing key; the parameter loader refuses a key that resembles a block it does not recognise. Each replaced a silent wrong answer with a loud refusal.

Make the invisible countable. Audit-write failures, readings discarded because a queue was full, buildings evicted from the detector's state, hashes below the current cost — each is a number on a screen precisely because its absence was previously indistinguishable from health.

Let the measurement contradict the document. Four of this project's published claims were corrected by its own harness. None of those corrections would have happened if the harness reported only what already worked.

Chapter 18

Limitations and future work

18.1 Limitations, stated as the screens state them

Each of these appears in the interface beside the number it qualifies, not only here.

1. **Lifetime carbon is an estimate.** A projection over the horizon in the parameters, not a measurement of anything that has happened.
2. **Integrity is not accuracy.** A verified chain says nobody altered what the meter sent. Whether the meter itself behaved is a separate question, and the one the alert inbox answers.
3. **The data is simulated.** Fifty real, named buildings, a real street network and a real catalogue structure, driven by a simulator rather than by fifty real meters. A replay path for a public smart-meter dataset is implemented and tested but is not what the live demonstration runs.
4. **One input is a labelled placeholder** — the 24-hour marginal emission profile — **and two are market estimates** inside a documented bracket, being the two solar cost terms. All three are marked as such wherever they appear.
5. **The portfolio is drawn from public-amenity tags**, which in Egypt do not mark a private school as private or a professional syndicate as anything but a public building. Exclusions are by words in the name, which is blunt in both directions: it would keep a private school whose name does not say so, and drop a state school calling itself an academy.
6. **Districts are administrative units, not neighbourhood names.** Each is whatever the geocoder resolves that building's own centroid to at city level, so a building a Cairene would place in Maadi is recorded under Helwan — the boundary it actually falls inside.
7. **No per-building forecast error figure is published.** The platform reports which model was selected and draws both lines rather than inventing a metric it does not measure.
8. **Three touch targets are under the 44px AAA floor** — 28, 42 and 40px. All three clear the 24px WCAG 2.5.8 minimum and each was left short deliberately, for the reasons in §9.
9. **The grid factor is constant over a thirty-year horizon**, which is questionable given Egypt's renewable build-out. A declining trajectory would reduce the credited benefit of every long-life measure. This is a parameter change rather than a code change, and it is named as an open question in the parameter file itself.

18.2 Threats to validity

The portfolio is real and named, but the readings are simulated, so results characterise the *decision model* rather than the buildings' actual behaviour. The saving fractions are literature

values rather than measured retrofit outcomes for these buildings, so absolute savings inherit that uncertainty — while the *relative* ranking, which is what the platform exists to produce, is far less sensitive to it.

The asymmetry in that last sentence is the honest defence of the whole project.

If every saving fraction in the catalogue were twenty per cent optimistic, every absolute figure this report quotes would be twenty per cent optimistic — and the funded set would be almost unchanged, because the optimiser ranks options against each other. That is why the claims are written about *comparisons* (this solver against that baseline, this objective against that one) rather than about absolute carbon. A claim of the form “this allocation avoids X tonnes” would rest entirely on the catalogue; a claim of the form “exact optimisation beats the repaired heuristic by $Y\%$ under a district cap” rests on the model.

18.3 Technologies evaluated and rejected

Each removal is a decision with a reason rather than an omission, and each is recorded at the module that resolves it.

Rejected	Why, and what replaced it
A managed public-cloud deployment	Abandoned after repeated free-tier account suspensions destroyed the environment more than once. One Compose file, rebuildable on any host in minutes — which also serves the portability argument the project makes anyway.
A general-purpose linear programming layer	Replaced by the constraint solver directly. The model is a multiple-choice knapsack with integral data; a constraint solver expresses the side constraints natively and proves optimality on every instance.
An additive time-series forecasting package	Two grounds: a compiled probabilistic backend that materially inflates image size and build time, and weak performance on sub-daily building load. A gradient-boosted regressor already in the dependency set outperformed it on the structure that dominates this problem.
A browser mapping library and a charting library	Both would need a runtime fetch or a bundling step, and the offline guarantee forbids both. The map and the charts are inline SVG.
A separate dashboard product	Removed when its panels moved into the application. It meant a second authentication surface, a second set of credentials, and panels querying the database with their own SQL that could disagree with the application about what a number meant.
A VPN tier for physical sensors	Specified for hardware that was never attached. Described as <i>designed for</i> rather than <i>built</i> , with the MQTT topic contract as the evidence that the extension point is real.
An SMTP alerting service	Nobody reads email during a presentation. Alerting is a row, a map marker and an optional outbound webhook.
Six independently deployed services	Collapsed to a modular monolith. The module boundaries still match the logical architecture, and the domain layer’s freedom from infrastructure is what lets thirteen claims be measured with no containers running.

A rejection is only honest if its cost is stated too.

Collapsing to one process means the throttle and the ingester are per-process state, which is why the application refuses to start under more than one worker — horizontal scaling now means more containers rather than more workers, and that is a real constraint rather than a free win. Dropping the mapping library means the map’s projection, pan and zoom are this project’s code to maintain. Dropping the charting library means the same for tick placement and label spacing, which is exactly where a resize defect hid for weeks. The offline guarantee was worth all three; it was not free.

18.4 Future work

18.4.1 Closing the layers that are shallower than they look

Per-end-use time-of-use weighting. The effective grid factor is applied once per *building* rather than per end use, even though the per-end-use breakdown already reaches the benefit function. Buildings therefore re-rank against each other — which is enough for the funded set to move — but measures *within* a building do not, so a generation option cannot flip against a fabric one on that path. The seam is in place and documented; the work is to use it.

A sourced marginal profile. The single change that would most improve the credibility of the time-of-use result is a published Egyptian hourly marginal emission series. Section 4 records what was searched so the next person does not repeat it.

Re-stepping the three severity colours so they separate for a deuteranopic reader without the hatch textures that currently stand in. The workaround is labelled as a workaround in the code.

18.4.2 Operational

Paging on the alert feed, which needs one parameter on the endpoint and is the last thing between that screen and a workable triage queue. Automatic refresh of the summary, of which the honest half — stating when the numbers were read — is already done. Digest pinning for pipeline actions. Enabling repository-level alerting so the dependency bot reaches somebody. Key-only authentication and automatic patching on the host.

18.4.3 What would make it a product rather than a demonstration

Three things, in order of how much they would change the argument.

Real meters. The MQTT topic is the seam and nothing downstream can tell a physical node from a simulated one, so this is an integration exercise rather than an architectural one — but every number in this report would change from characterising the model to characterising the buildings.

Measurement and verification after the fact. The platform predicts a saving and never learns whether it happened. A retrofit completed is a natural experiment, and comparing the post-retrofit series against the counterfactual the forecaster already produces would turn the catalogue’s literature values into measured ones — which is the single largest source of uncertainty named in §18.3 and in the limitations above.

Multi-tenancy. The seam exists: an organisation table with exactly one row, and users already belonging somewhere, so scoping the portfolio later is additive rather than structural. It was not built because full multi-tenancy means an identifier on every table and a filter on every query, and the failure mode of getting one query wrong is showing one authority another authority’s portfolio. That is not a change to rush.

Appendix A

Appendices

A.1 Reproducing every number in this report

Every figure in Chapter 6 is regenerated by one command against the repository’s own data directory, with no database and no containers:

```
| python -m gemp.evaluate
```

It writes the claims and the solver sweep to `out/`, and exits non-zero if any claim has stopped holding. The database-backed results of §6.7 additionally require the stack and a completed refit (`--with-db`).

Command	What it does
<code>python scripts/check.py</code>	the four gates, exactly as the pipeline runs them
<code>python -m gemp.domain.catalog --validate</code>	the data contract; <code>--strict</code> additionally fails on an uncited row
<code>python -m gemp.evaluate</code>	re-measure every claim, about 20 s
<code>python -m gemp.evaluate --with-db</code>	adds forecasting, anomaly and integrity
<code>python -m gemp.optimize.cli --budget N --compare</code>	solve and print an allocation across all solvers
<code>python -m gemp.seed --months 6</code>	backfill signed history
<code>python -m gemp.ml.jobs</code>	refit every forecaster, rewrite the annual figures and load shapes
<code>python -m gemp.ml.evaluate --k 4 6 8 10</code>	sweep the anomaly threshold
<code>python -m gemp.sim.replay --stats-only</code>	report what the real meter dataset looks like against a building
<code>python -m gemp.auth.bootstrap --username you</code>	create the first administrator; prints a password once
<code>make security</code>	the identical scanner set the pipeline runs
<code>make backup && make backup-verify</code>	back up, then restore into a scratch database and assert on it

There is no default account and no default password, by design.

A.2 Configuration reference

Every setting, with the ones that are security-relevant marked. All are read from the environment with a file fallback, under a common prefix.

Key	Default	Meaning
HMAC_KEY	—	Required. The integrity signing key; no default exists, and a system whose guarantee silently depends on a well-known development key is worse than one that refuses to start
DB_*, MQTT_*	—	credentials; no defaults
SIM_SPEED	720	data-seconds per wall-second
SIM_CLAMP_TO_WALL_CLOCK	true	spend the acceleration only on catching up
COOKIE_SECURE	false	must be true on anything reachable over a network; the administration screen reports it
API_DOCS	false	the interactive schema is a reconnaissance map; off unless somebody opens it, and even then not anonymous
HTTP_BIND / HTTP_PORT	127.0.0.1 / 8080	binding beyond loopback requires the secure-cookie flag
WEBHOOK_URL	unset	environment only — a notification target any caller could change is an exfiltration primitive
WEBHOOK_MIN_SEVERITY	high	“high” rather than “critical” so a stuck meter is forwarded
JUDGE_PASS_CODE	unset	empty disables the feature entirely
JUDGE_PASS_UNTIL	unset	must carry a UTC offset or passes stay off
JUDGE_PASS_DAILY_MAX	90	ceiling per rolling 24 hours
BREVO/RESEND/SMTP_*	unset	first configured provider wins; unset means the pass still works and only the email is skipped
INGEST_ENABLED, SCHEDULER_ENABLED	1	for running the API without its background threads
WEB_CONCURRENCY	—	greater than one refuses to start

A.3 Data model

Table	Key	Holds
building	id	the portfolio, including the footprint as JSON, the annualised consumption and its source, and the measured hour-of-day load shape
reading	(building, ts)	the hypertable: kW, source, sequence and signature. Chunked at 30 days of data time
integrity_checkpoint	(building, ts)	anchored chain heads, mirrored outside the volume
intervention	id	a snapshot of the catalogue, so a stored run remains reproducible against the catalogue as it was
candidate	key	materialised options with the hash of the inputs that produced them; deliberately not on the request path
forecast	(building, ts)	one forecast per building per timestamp, replaced rather than accumulated
anomaly	id	observed, expected, residual, robust score, severity, acknowledged
optimization_run	id	budget, objective, solver, status, cap, totals, solve time and the full input hash
allocation	id	the funded items of a run
organization	id	the tenancy seam, one row
app_user	id	username, role, script hash, forced-change flag, active flag, and an optional access end date
user_session	token hash	never the token; created, last seen, expires, revoked, address, agent
audit_log	id	who did what, when, to what, and the outcome — including denials
judge_pass	id	the address a pass was issued to and what became of its email
app_setting	key	switches an administrator flips, in the database so they survive a restart

Four migrations, applied forward only: the initial schema; identity, access and audit; the building load shape; and the Judge Pass. The TimescaleDB objects — the hypertable and the hourly continuous aggregate — live outside the migration tool, because a continuous aggregate cannot be created inside a transaction.

A.4 The claims, with their gates

ID	Gate	Why that number
F1-a	no loss beyond 10^{-9}	a loss bigger than float summation order means the baselines are not solving the same problem
F1-b	identical optimum	pruning is sold as result-preserving
F1-c	set identity	expansion must be a pure function of its inputs
F4	strict sub-additivity	only where an end use is shared; no bundle may exceed the sum of its parts
F5	shortfall $\leq 1.0\%$	measured as what the difference is <i>worth</i> , not as set identity — near a boundary the objectives swap options of nearly equal value
F7	more than one winner	the claim is diversity, not the absence of any one measure
F8-a	median $\leq 25\%$	the sentence “a good heuristic is near-optimal here” survives up to a quarter
F8-b	capped $\geq 3\times$ uncapped	the claim is the <i>ratio</i> ; without it the argument for an exact solver is rhetorical
F8-c	100 % proven optimal	“proven optimum” is a claim about status, not objective
F8-d	zero losses	against the do-nothing-clever policy
F8-e	zero violations	more money must not buy less benefit; any violation is a bug
F8-f	a plateau exists	keeps the saturation finding visible so nobody requotes the old headline
F15	≥ 1 changed budget	one changed budget proves the channel is live; the count is the finding
F3	beats the baseline	a forecaster that cannot beat “same hour last week” has not earned its runtime
F9-a	recall ≥ 0.80	the review’s gate
F9-b	precision ≥ 0.60	the review’s gate
F5-b	chains match the anchor	a chain always verifies against itself, including after its tail is deleted
DATA	every row cited	every number the platform reports derives from six rows

A.5 Repository inventory

174 tracked files, roughly 42 300 lines.

Path	Files	Contents
src/gemp/	62	the application, in nine packages
tests/	28	548 collected cases
web/	18	nine screens, one stylesheet, the basemap and street context
infra/	17	container build, compose, nginx, broker, database init, six deploy scripts
docs/	11	pipeline, server, findings, screenshots
scripts/	9	portfolio and basemap fetchers, environment and broker setup, the gate script, the local gate run
migrations/	6	four revisions plus the environment
.github/	6	four workflows, code owners, dependency configuration
data/	4	the portfolio, the catalogue, the parameters, and the contract that governs them

A.6 Sources for the parameters and the catalogue

- [1] Egyptian Electricity Holding Company, *Annual Report 2023/2024*. Cairo, 2024. Carbon dioxide emissions of thermal generating plants, p. 30; maximum-load-day curve, p. 15.
- [2] Egyptian Electric Utility and Consumer Protection Regulatory Agency, *Electricity Tariff Starting From April 2026*. Low voltage (380 V), “Other Users”.

- [3] European Commission Joint Research Centre, *Photovoltaic Geographical Information System*, version 5.3, radiation database PVGIS-SARAH3.
- [4] ASHRAE, *Handbook — Fundamentals*, 2021, chapter 15, “Fenestration”.
- [5] N. Fernandez et al., *Impacts of Commercial Building Controls on Energy Savings and Peak Load Reduction*, PNNL-25985, 2017 (revised October 2017).
- [6] S. Saadatian, F. Freire and N. Simões, “Embodied impacts of window systems: a comparative assessment of framing and glazing alternatives”, *Journal of Building Engineering* 35:102042, 2021.
- [7] A. S. Mohamed, B. M. Gomaa and A. E. N. Sarhan, “Assessment of buildings’ energy-saving strategies resilience to climate change: the case of Mediterranean climate”, *HBRC Journal* 22(1), article 41, 2026.
- [8] International Energy Agency, *The Future of Cooling*. Paris, 2018.
- [9] Fraunhofer Institute for Solar Energy Systems ISE, *Photovoltaics Report*. Freiburg, 2024, p. 37.
- [10] International Renewable Energy Agency, *Renewable Power Generation Costs in 2024*. Abu Dhabi, 2025.
- [11] International Renewable Energy Agency, *Renewable Energy Outlook: Egypt*. Abu Dhabi, 2018. *Checked and found to contain no hourly series*.
- [12] UNFCCC Clean Development Mechanism, *Tool to calculate the emission factor for an electricity system* (AM-TOOL-07). *Annual rather than hourly*.
- [13] Institute for Global Environmental Strategies, *List of Grid Emission Factors*. *Annual rather than hourly*.
- [14] OpenStreetMap contributors, via the Overpass and Nominatim services. Building footprints, names and districts, under the Open Database Licence.
- [15] UCI Machine Learning Repository, *Individual Household Electric Power Consumption Data Set*.
- [16] Sentinel-2 cloudless 2020 by EOX IT Services GmbH (Copernicus data), CC BY 4.0.

Entries 11 to 13 are listed deliberately: they are sources that were checked and found *not* to support the value they were sought for, and recording them is what stops the search being repeated.